Points of Interest as Crime Generators:

A Supervised Regression Analysis of London Ward-Level Crime Density

> I, Emily Deeb, pledge my honour that the work presented in this assessment is my own. Where information has been derived from other sources, I confirm that this has been indicated in the work.
>
> **Student Number:** 25127081

Preparation

- Github link
- Number of words: 1,500
- Runtime: *** hours (*Memory 10 GB, CPU Intel i7-10700 CPU @2.90GHz*)
- Coding environment: SDS Docker (or anything else)
- License: this notebook is made available under the Creative Commons Attribution license (or other license that you like).
- Additional library *[libraries not included in SDS Docker or not used in this module]*:
  - **watermark**: A Jupyter Notebook extension for printing timestamps, version numbers, and hardware information.
  - ……

```
# Step 01: Imports + BASE URL
import pandas as pd
import geopandas as gpd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import seaborn as sns
import osmnx as ox
import tempfile, os, requests

BASE = "https://raw.githubusercontent.com/EmilyDeeb/0006_Assessment/main/data/"
```

Table of contents

1. Introduction
2. Research questions
3. Data
4. Methodology
5. Results and discussion
6. Conclusion
7. References

Introduction

[go back to the top]

This study examines the spatial relationship between urban amenity density and crime concentration across London wards. We use open-source point-of-interest (POI) data from OpenStreetMap and recorded crime data from the Metropolitan Police Service within a supervised regression model to assess whether the spatial layout of urban amenities such as cafés, clubs, and restaurants predicts crime density and whether specific amenity types are associated with different crime categories.

Routine Activity Theory (Cohen and Felson, 1979) argues that crime occurs when motivated offenders, suitable targets, and the absence of capable guardians intersect in the same place and time. This convergence can be defined by the functional land-use profile of an area; the mix of amenities, services, and commercial activity that determines who is present and when. Brantingham and Brantingham (1995) formalise this with the concept of crime generators, theorising that locations that attract large numbers of people for non-criminal reasons increase crime opportunities by providing many potential offenders and targets.

Point-of-interest (POI) data has become an important resource in empirical research on urban crime. Cichosz (2020) demonstrated that OSM POI density features are highly effective at predicting crime risk across UK urban areas, surpassing traditional land-use classifications. POI data also serves as a proxy for ambient population: Andresen (2011) argued that the number of people actually present, rather than the residential population, is a more meaningful denominator for calculating crime rates. This methodological distinction is crucial, as Malleson and Andresen (2016) showed for London that relying on residential population data generates misleading hotspots for non-residential crimes such as theft from the person.

Although significant progress has been made in analyzing the relationship between crime and urban fabric, most studies continue to model total crime or individual crime types in isolation. The extent to which different POI categories generate distinct crime types, or whether urban

density drives crime uniformly across categories, remains underexplored, especially at the ward level in London. To address this gap, we split the crime dataset into two types; Type A (person/society crime) and Type B (property crime), and analysed these alongside POI categories. This study is among the first to apply SHAP decomposition to compare crime generator profiles across multiple crime categories simultaneously at ward level in London.

Research question

[go back to the top]

Primary: To what extent does the density of urban amenities predict person/society and property crime densities across London wards?

Sub-question: Which specific amenity categories serve as the primary crime generators for each crime type, and do these differ systematically between person/society and property crime?

Data and Feature Engineering

[go back to the top]

Three datasets were integrated for this analysis.

Data 01: Metropolitan Police Service recorded crime data were obtained from the London Datastore (data.london.gov.uk), containing monthly crime counts by ward and crime category for the 24 months to February 2025. Crimes were summed across the full period and aggregated into two theoretically motivated categories:

Table 1: Crime category definitions and offence types included in the analysis

Crime Category	Type	Offence Types Included
Type A — Person/Society Crime	Target variable	Violence against the person, robbery, sexual offences, possession of weapons, drug offences, public order offences, miscellaneous crimes against society, fraude and forgery
Type B — Property Crime	Target variable	Theft, burglary, vehicle offences, arson and criminal damage

Note: NFIB fraud offences were excluded from both categories as fraud is recorded at the victim's home address rather than the location of the offence, rendering it spatially uninformative at the ward level.

```
# Step 02: Load crime → aggregate Type A + Type B → crime densities
crime_raw = pd.read_csv(BASE + "crime.csv")

# Type A - Person/Society crime categories
typeA_cats = [
    'VIOLENCE AGAINST THE PERSON', 'ROBBERY',
    'SEXUAL OFFENCES', 'POSSESSION OF WEAPONS',
    'DRUG OFFENCES', 'PUBLIC ORDER OFFENCES',
    'MISCELLANEOUS CRIMES AGAINST SOCIETY', 'FRAUD AND FORGERY'
]

# Type B - Property crime
typeB_cats = [
    'THEFT', 'BURGLARY',
    'VEHICLE OFFENCES', 'ARSON AND CRIMINAL DAMAGE'
]
```

```
# Sum monthly columns to annual count
month_cols = [c for c in crime_raw.columns if str(c).startswith("2")]
crime_raw["annual_count"] = crime_raw[month_cols].sum(axis=1)

def aggregate_crime(df, categories, label):
    """Filter to crime categories and sum to ward level."""
    return (
        df[df["MajorText"].isin(categories)]
        .groupby(["WardCode", "WardName", "LookUp_BoroughName"])["annual_count"]
        .sum()
        .reset_index()
        .rename(columns={
            "annual_count": label,
            "LookUp_BoroughName": "Borough"
        })
    )

crime_A = aggregate_crime(crime_raw, typeA_cats, "typeA_count")
crime_B = aggregate_crime(crime_raw, typeB_cats, "typeB_count")

ward_crime = crime_A.merge(
    crime_B[["WardCode", "typeB_count"]], on="WardCode", how="left"
```

```
).fillna(0)

print(f"Wards: {len(ward_crime)}")
ward_crime.head()
```

```
Wards: 679
```

	WardCode	WardName	Borough	typeA_count	typeB_count
0	E05009317	Bethnal Green	Tower Hamlets	2811	2403
1	E05009318	Blackwall & Cubitt Town	Tower Hamlets	1776	1615
2	E05009319	Bow East	Tower Hamlets	2085	1973
3	E05009320	Bow West	Tower Hamlets	1422	1457
4	E05009321	Bromley North	Tower Hamlets	1482	1403

```
fig, ax = plt.subplots(figsize=(7, 4.5), facecolor='#F7F4ED')
ax.set_facecolor("#F7F4ED")

bars = ax.bar(
    ['Type A - Person/Society', 'Type B - Property'],
    [ward_crime['typeA_count'].sum(), ward_crime['typeB_count'].sum()],
    color=['#A35139', '#2C3B4D'],
    width=0.45,
    edgecolor='none'
)

# Value labels on top of bars
for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height + 5000,
        f'{height:,.0f}',
        ha='center', va='bottom',
        fontsize=11, fontweight='500',
        color='#1B2632',
        fontfamily='sans-serif'
    )

ax.set_ylabel('Total recorded crimes (24 months)',
              fontsize=11, color='#1B2632', labelpad=10)
```

```
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:,.0f}'))
ax.tick_params(colors='#1B2632', labelsize=10)
ax.spines[['top', 'right']].set_visible(False)
ax.spines[['left', 'bottom']].set_color('#C9C1B1')
ax.yaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
ax.set_axisbelow(True)

# Remove ax.set_title entirely
# Add everything as fig.text below

fig.text(0.5, -0.01,

         '\n Source: Metropolitan Police Service, London Datastore (data.london.gov.uk).',
         ha='center', fontsize=9, color='#615D56', style='italic',
         wrap=True)

plt.tight_layout()
plt.subplots_adjust(bottom=0.12)
plt.show()
```

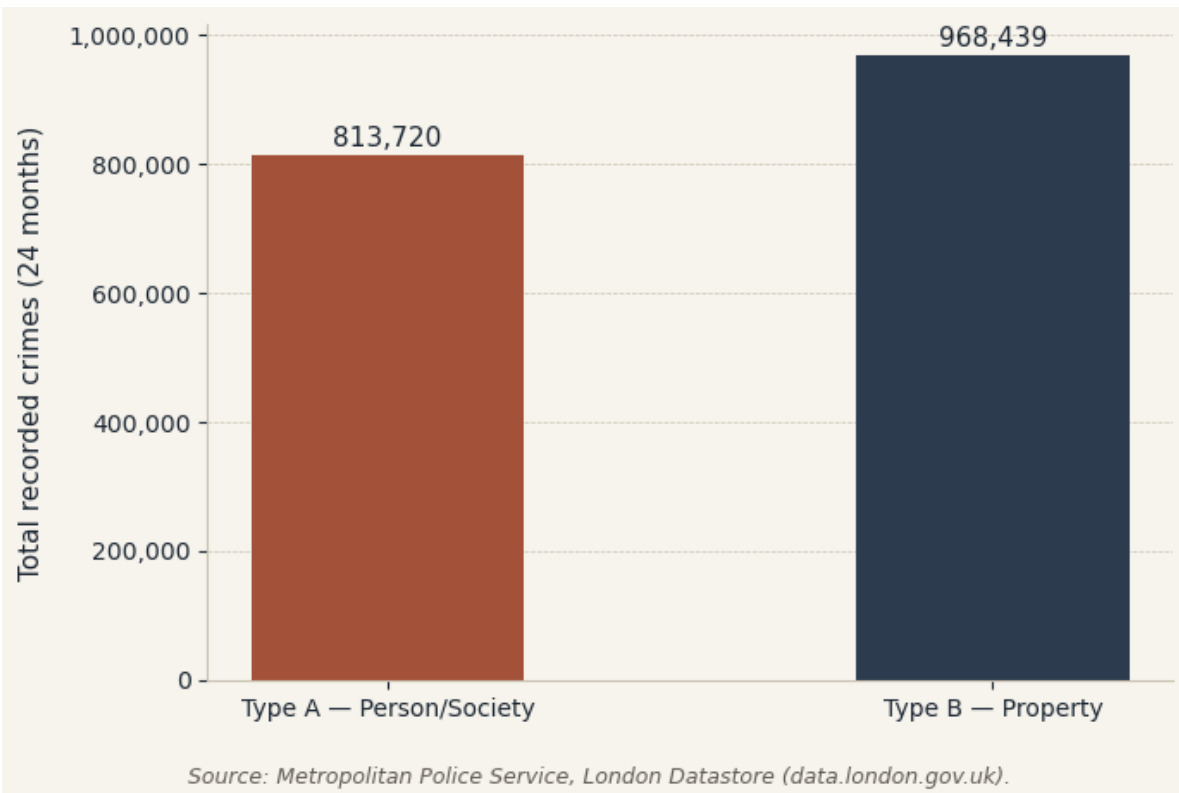


Figure 1: Total recorded crimes by category across 679 London wards, 24 months to February 2025. *Notes: Type B (property crime) exceeds Type A (person/society crime) by approximately 19%.*

Data 02: Ward Spatial Boundary and population

Table 2: Spatial and demographic data sources

Dataset	Source	URL	Geography	Notes
Ward boundary polygons	ONS Open Geography Portal	geoportal.statistics.gov.uk	Ward level	2024 boundaries, filtered to London using GLA boundary
Ward areas	Derived from boundary polygons	—	Ward level	Calculated using British National Grid projection (EPSG:27700)
Population estimates	ONS mid-2021 ward-level estimates	geoportal.statistics.gov.uk	Ward level	No missing values after merging; 679 London wards retained

Note: Population density was retained in the dataset but excluded from model features to isolate the contribution of amenity configuration independently of population size. Incorporating it as a control variable is a natural next step.

```
# Step 03: Load population
pop = pd.read_csv(BASE + "ward_population.csv", encoding="latin-1")

pop = pop.rename(columns={
    "Ward 2023 Code": "WardCode",
    "Total": "population"
})[["WardCode", "population"]]

pop["population"] = (
    pop["population"].astype(str)
    .str.replace(",", "")
    .astype(int)
)

pop = pop[pop["WardCode"].str.startswith("E05")].copy()
```

```
ward_crime = ward_crime.merge(pop, on="WardCode", how="left")
print(f"Missing population: {ward_crime['population'].isna().sum()}")
```

```
Missing population: 0
```

```
# Step 04: Load GLA boundary + filter 2024 wards to London
# GLA boundary
gla_base = "https://raw.githubusercontent.com/EmilyDeeb/0006_Assessment/main/data/gla/"
tmp_gla = tempfile.mkdtemp()
for ext in ["shp", "dbf", "shx", "prj", "cpg"]:
    fname = f"London_GLA_Boundary.{ext}"
    r = requests.get(gla_base + fname)
    with open(os.path.join(tmp_gla, fname), "wb") as f:
        f.write(r.content)

gla_gdf = gpd.read_file(os.path.join(tmp_gla, "London_GLA_Boundary.shp"))
gla_gdf = gla_gdf.to_crs("EPSG:4326")

# 2024 ward boundaries - filter to London via GLA polygon
wards_2024 = gpd.read_file(BASE + "wards_2024.geojson").to_crs("EPSG:4326")

# Use projected CRS for centroid calculation
wards_proj = wards_2024.to_crs("EPSG:27700")
wards_2024["centroid"] = wards_proj.geometry.centroid.to_crs("EPSG:4326")
wards_centroids = gpd.GeoDataFrame(
    wards_2024, geometry="centroid", crs="EPSG:4326"
)

london_wards = gpd.sjoin(
    wards_centroids,
    gla_gdf[["geometry"]],
    how="inner", predicate="within"
)
london_wards = wards_2024.loc[london_wards.index].copy()
london_wards = london_wards.rename(columns={
    "WD24CD": "WardCode",
    "WD24NM": "WardName_geo"
})
london_wards["area_km2"] = (
    london_wards.to_crs("EPSG:27700").geometry.area / 1_000_000
)
```

```
london_wards[['WardCode', 'WardName_geo', 'area_km2']].head()
```

	WardCode	WardName_geo	area_km2
1104	E05009288	Aldersgate	0.098405
1105	E05009289	Aldgate	0.114784
1106	E05009290	Bassishaw	0.106979
1107	E05009291	Billingsgate	0.086042
1108	E05009292	Bishopsgate	0.254265

```
#   Step 05: Build main GeoDataFrame

# Merge crime counts with ward geometries
ward_gdf = ward_crime.merge(
    london_wards[["WardCode", "area_km2", "geometry"]],
    on="WardCode",
    how="left",
)

# Compute crime density (target variables)
ward_gdf["typeA_density"] = ward_gdf["typeA_count"] / ward_gdf["area_km2"]
ward_gdf["typeB_density"] = ward_gdf["typeB_count"] / ward_gdf["area_km2"]

# Promote to GeoDataFrame and drop rows with missing spatial data
ward_gdf = (
    gpd.GeoDataFrame(ward_gdf, geometry="geometry", crs="EPSG:4326")
    .dropna(subset=["geometry", "area_km2"])
)

print(f"Final ward count:    {len(ward_gdf)}")
print(f"Missing geometries:  {ward_gdf['geometry'].isna().sum()}")

#   Styled summary: crime density descriptive stats (pre-transformation)

STYLE = {
    "font-family": "Inter, sans-serif",
    "font-size":   "12px",
    "color":       "#1B2632",
    "border-bottom": "1px solid #C9C1B1",
    "padding":     "7px 12px",
}
```

```
TABLE_STYLES = [
    {
        "selector": "thead tr",
        "props": [
            ("background-color", "#2C3B4D"),
            ("color",            "#EEE9DF"),
            ("font-weight",      "500"),
        ],
    },
    {
        "selector": "caption",
        "props": [
            ("font-size",   "11px"),
            ("color",       "#C9C1B1"),
            ("font-style",  "italic"),
            ("text-align",  "left"),
            ("padding",     "6px 0"),
        ],
    },
]

(
    ward_gdf[["typeA_density", "typeB_density"]]
    .describe()
    .round(2)
    .style
    .format("{:.2f}")
    .set_properties(**STYLE)
    .set_table_styles(TABLE_STYLES)
)
```

```
Final ward count:    679
Missing geometries:  0
```

Table 5

	typeA_density	typeB_density
count	679.00	679.00
mean	868.35	1053.34
std	765.14	1505.26

	typeA_density	typeB_density
min	8.47	8.56
25%	322.69	304.79
50%	619.01	596.75
75%	1199.32	1237.23
max	4978.44	24964.44

Table 3: Crime density descriptive statistics (crimes per km^2) — pre-transformation. *Notes: Pre-transformation density distributions show strong right skew in both target variables. The standard deviation exceeds the mean for both* ***typeA_density*** *(=765, =868) and* ***typeB_density*** *(=1505, =1053), with maximum values far exceeding the 75th percentile, indicating a small number of high-density outlier wards.* ***typeB_density*** *exhibits notably greater dispersion. Log transformation will be applied prior to modelling.*

Data 03: Point-of-interest data were extracted from OpenStreetMap using the osmnx Python library (Boeing, 2017) and queried within the Greater London Authority boundary polygon. After cleaning, 28,059 POIs were retained across 18 categories. The table below shows the categories, OSM tags used for extraction, and counts:

```
# Step 06: Download POIs from OpenStreetMap
gla_polygon = gla_gdf.to_crs("EPSG:4326").union_all()

tags = {
    "amenity": ["pub", "bar", "nightclub", "fast_food", "restaurant",
                "cafe", "bank", "atm", "pharmacy"],
    "railway": ["station"],
    "shop": ["supermarket", "convenience", "mall", "department_store",
             "clothes", "electronics", "jewelry"]
}

print("Downloading POIs - this takes 5-10 minutes...")
poi_raw = ox.features.features_from_polygon(gla_polygon, tags=tags)

# Include both points AND polygon centroids
# Many venues (pubs, restaurants) are mapped as building polygons in OSM
poi_points = poi_raw[poi_raw.geometry.type == "Point"].copy()
poi_polys  = poi_raw[poi_raw.geometry.geom_type.isin(["Polygon", "MultiPolygon"])].copy()
poi_polys["geometry"] = poi_polys.to_crs("EPSG:27700").geometry.centroid.to_crs("EPSG:4326")

poi_gdf = pd.concat([poi_points, poi_polys], ignore_index=True)

# Extract poi_type - check amenity, shop, railway in priority order
```

```python
poi_gdf["poi_type"] = (
    poi_gdf["amenity"]
    .fillna(poi_gdf.get("shop", pd.NA))
    .fillna(poi_gdf.get("railway", pd.NA))
)

poi_gdf = poi_gdf[["geometry", "poi_type"]].dropna()
poi_gdf = poi_gdf.set_crs("EPSG:4326")

print(f"Total POIs (points + polygon centroids): {len(poi_gdf)}")
# print(poi_gdf["poi_type"].value_counts())
# We have extra points because some venues are mapped as both points and polygons in OSM. We
# remove doctors
```

```
Downloading POIs - this takes 5-10 minutes...
Total POIs (points + polygon centroids): 41523
```

```python
# Step 06b/c: Filter noise → spatial join → densities
valid_types = [
    "pub", "bar", "nightclub", "fast_food", "restaurant", "cafe",
    "bank", "atm", "pharmacy", "station",
    "supermarket", "convenience", "mall", "department_store",
    "clothes", "electronics", "jewelry"
]

poi_gdf = poi_gdf[poi_gdf["poi_type"].isin(valid_types)].copy()
print(f"POIs after cleaning: {len(poi_gdf)}")

wards_4326 = ward_gdf[["WardCode", "geometry"]].to_crs("EPSG:4326")
poi_in_wards = gpd.sjoin(poi_gdf, wards_4326, how="left", predicate="within")

poi_counts = (
    poi_in_wards
    .groupby(["WardCode", "poi_type"])
    .size()
    .unstack(fill_value=0)
    .reset_index()
)
poi_counts.columns.name = None

# Drop existing POI/density columns if re-running
existing = [c for c in poi_counts.columns if c != "WardCode" and c in ward_gdf.columns]
```

```
existing += [c for c in ward_gdf.columns if c.endswith("_density")
             and c not in ["typeA_density", "typeB_density"]]
ward_gdf = ward_gdf.drop(columns=existing, errors="ignore")

# Merge - left join preserves all 679 wards
ward_gdf = ward_gdf.merge(poi_counts, on="WardCode", how="left")

# Ensure columns exist for wards with no POIs
poi_cols = [c for c in poi_counts.columns if c != "WardCode"]
for col in poi_cols:
    if col not in ward_gdf.columns:
        ward_gdf[col] = 0
ward_gdf[poi_cols] = ward_gdf[poi_cols].fillna(0)

# Compute densities
for col in poi_cols:
    ward_gdf[f"{col}_density"] = ward_gdf[col] / ward_gdf["area_km2"]

poi_density_cols = [f"{c}_density" for c in poi_cols]

# Report empty wards
empty = ward_gdf[ward_gdf[poi_cols].sum(axis=1) == 0][["WardCode", "WardName", "Borough"]]
print(f"Wards with no POIs: {len(empty)}")
print(empty.to_string(index=False))
print(f"\nPOI density features: {len(poi_density_cols)}")
```

```
POIs after cleaning: 41502
Wards with no POIs: 0
Empty DataFrame
Columns: [WardCode, WardName, Borough]
Index: []

POI density features: 17
```

```
# Table 3: POI categories, OSM tags, and counts
tag_map = {
    'restaurant':       ('Restaurant',        'amenity=restaurant'),
    'cafe':             ('Cafe',              'amenity=cafe'),
    'fast_food':        ('Fast food',         'amenity=fast_food'),
    'convenience':      ('Convenience store', 'shop=convenience'),
    'clothes':          ('Clothes shop',      'shop=clothes'),
```

```
    'atm':              ('ATM',               'amenity=atm'),
    'pub':              ('Pub',               'amenity=pub'),
    'pharmacy':         ('Pharmacy',          'amenity=pharmacy'),
    'bar':              ('Bar',               'amenity=bar'),
    'supermarket':      ('Supermarket',       'shop=supermarket'),
    'station':          ('Rail station',      'railway=station'),
    'bank':             ('Bank',              'amenity=bank'),
    'jewelry':          ('Jewellery shop',    'shop=jewelry'),
    'electronics':      ('Electronics shop',  'shop=electronics'),
    'nightclub':        ('Nightclub',         'amenity=nightclub'),
    'department_store': ('Department store',  'shop=department_store'),
    'mall':             ('Mall',              'shop=mall'),
}

counts = poi_gdf["poi_type"].value_counts()

rows = []
for poi_type, (label, tag) in tag_map.items():
    count = counts.get(poi_type, 0)
    rows.append({'POI Category': label, 'OSM Tag': tag, 'Count': int(count)})

table3 = pd.DataFrame(rows).sort_values('Count', ascending=False).reset_index(drop=True)

total_row = pd.DataFrame([{
    'POI Category': 'Total',
    'OSM Tag': '',
    'Count': table3['Count'].sum()
}])
table3 = pd.concat([table3, total_row], ignore_index=True)

(table3.style
    .format({'Count': '{:,.0f}'})
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '13px',
        'text-align': 'left',
        'color': '#1B2632',
        'background-color': '#F7F4ED',
        'border-bottom': '1px solid #C9C1B1',
        'padding': '8px 14px'
    })
    .set_table_styles([
```

```
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#EEE9DF'),
                   ('font-weight', '500'),
                   ('font-size', '13px')]},
        {'selector': 'tbody tr:nth-child(even)',
         'props': [('background-color', '#EAE6DE')]},
        {'selector': 'tr:last-child',
         'props': [('font-weight', 'bold'),
                   ('border-top', '2px solid #2C3B4D')]},
        {'selector': 'caption',
         'props': [('font-size', '11px'),
                   ('color', '#C9C1B1'),
                   ('font-style', 'italic'),
                   ('text-align', 'left'),
                   ('padding', '6px 0')]}
    ])
    .hide(axis='index')
)
```

Table 6

POI Category	OSM Tag	Count
Restaurant	amenity=restaurant	8,978
Cafe	amenity=cafe	6,718
Fast food	amenity=fast_food	6,158
Convenience store	shop=convenience	5,432
Pub	amenity=pub	3,177
Clothes shop	shop=clothes	2,933
ATM	amenity=atm	1,674
Pharmacy	amenity=pharmacy	1,280
Supermarket	shop=supermarket	1,252
Bar	amenity=bar	1,215
Bank	amenity=bank	703
Jewellery shop	shop=jewelry	695
Rail station	railway=station	616
Electronics shop	shop=electronics	300
Nightclub	amenity=nightclub	154
Department store	shop=department_store	134
Mall	shop=mall	83
Total		41,502

Table 4: Points of Interest (POI) Extracted from OpenStreetMap — Greater London *Notes: 41,502 POIs across 17 categories extracted via OSMnx. Food and beverage venues dominate (restaurants, cafes, fast food, pubs, bars — 26,246 combined, 63% of total). Retail and services account for the remainder. Categories were selected as proxies for footfall and economic activity in subsequent crime density modelling.*

```
# Step 06c: POI density descriptive statistics
from matplotlib.colors import LinearSegmentedColormap

palette_cmap = LinearSegmentedColormap.from_list(
    'palette', ['#F7F4ED', '#C9C1B1', "#A8696F"]
)

# Build describe table
desc = ward_gdf[poi_density_cols].describe().round(3).T

# Replace count with total POI count per category
desc['count'] = [ward_gdf[col.replace('_density', '')].sum()
                 for col in poi_density_cols]

(desc
    .drop(columns=['min'])
    .rename(columns={'count': 'Total POIs', 'mean': 'Mean', 'std': 'Std',
                     '25%': 'Q1', '50%': 'Median', '75%': 'Q3', 'max': 'Max'})
    .style
    .format({'Total POIs': '{:.0f}', 'Mean': '{:.3f}', 'Std': '{:.3f}',
             'Q1': '{:.3f}', 'Median': '{:.3f}', 'Q3': '{:.3f}', 'Max': '{:.3f}'})
    .background_gradient(subset=['Mean', 'Std', 'Max'], cmap=palette_cmap)
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '12px',
        'color': '#1B2632',
        'border-bottom': '1px solid #C9C1B1',
        'padding': '7px 12px'
    })
    .set_table_styles([
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#EEE9DF'),
                   ('font-weight', '500')]},
        {'selector': 'caption',
         'props': [('font-size', '11px'),
                   ('color', '#C9C1B1'),
```

```
                    ('font-style', 'italic'),
                    ('text-align', 'left'),
                    ('padding', '6px 0')]}
    ])
)
```

Table 7

	Total POIs	Mean	Std	Q1	Median	Q3	Max
atm_density	1629	1.866	2.908	0.000	0.739	2.332	20.721
bank_density	643	0.722	1.866	0.000	0.000	0.333	12.158
bar_density	1098	1.372	3.971	0.000	0.000	1.052	47.284
cafe_density	6482	7.582	11.638	0.978	3.226	9.320	99.362
clothes_density	2861	3.547	13.987	0.000	0.000	1.760	191.327
convenience_density	5392	6.256	6.350	1.621	3.904	9.195	35.078
department_store_density	133	0.132	0.483	0.000	0.000	0.000	5.052
electronics_density	300	0.338	0.839	0.000	0.000	0.252	8.769
fast_food_density	5909	6.664	9.457	1.154	3.086	8.116	61.386
jewelry_density	678	0.782	3.937	0.000	0.000	0.074	79.490
mall_density	81	0.102	0.390	0.000	0.000	0.000	3.155
nightclub_density	150	0.195	0.788	0.000	0.000	0.000	9.632
pharmacy_density	1268	1.377	1.662	0.314	0.882	1.900	14.568
pub_density	3047	3.472	5.393	0.575	1.375	4.091	46.971
restaurant_density	8686	10.235	19.567	1.015	3.485	11.093	235.109
station_density	599	0.600	0.858	0.000	0.305	0.914	5.022
supermarket_density	1246	1.333	1.785	0.000	0.708	1.845	13.470

Table 5: POI Density Statistics by Category — Ward-Level Distribution Across Greater London *Notes: Ward-level POI densities (POIs per km²) for 17 categories across 679 wards. Restaurant and cafe densities show the highest means and the most extreme skew — restaurant_density peaks at 235.1 and clothes_density at 191.3, far exceeding their respective Q3 values, confirming heavy concentration in a small number of central wards. Sparse categories (mall, department_store, nightclub) have a median of zero, indicating absence from the majority of wards. Log transformation will be applied to all density features prior to modelling.*

Feature Engineering: All crime and POI counts were normalised by ward area to produce density measures (per km²), accounting for variation in ward size across the 679 London wards. A log transformation was then applied to all density variables to address right skewness prior to modelling:

$$x' = \log\left(\frac{\text{count}_i}{\text{area}_i} + 1\right)$$

where x' is the transformed feature, count_i is the raw crime or POI count for ward i, area_i is the ward area in km^2, and the $+1$ constant prevents undefined values where counts are zero. After transformation, residual skewness fell below 1.5 for 14 of the 18 POI features, with only mall, department store, nightclub, and jewellery remaining moderately skewed due to their sparse distribution across wards.

```
# ================================================================
# CELL 11 - Distribution plots: raw vs log transformed
# ================================================================
fig, axes = plt.subplots(2, 2, figsize=(12, 8), facecolor='#F7F4ED')

pairs = [
    ("typeA_density", "typeA_density_log",
     "Type A - raw (crimes per km²)", "Type A - log transformed"),
    ("typeB_density", "typeB_density_log",
     "Type B - raw (crimes per km²)", "Type B - log transformed"),
]

for row, (raw, log, raw_title, log_title) in enumerate(pairs):
    for col, (col_name, title, color) in enumerate([
        (raw, raw_title, '#A35139'),
        (log, log_title, '#2C3B4D')
    ]):
        ax = axes[row][col]
        ax.set_facecolor('#F7F4ED')

        feature_matrix[col_name].hist(
            bins=40, ax=ax,
            color=color, edgecolor='#F7F4ED', linewidth=0.3
        )

        ax.set_title(title, fontsize=11, fontweight='bold',
                     color='#1B2632', pad=8)
        ax.set_xlabel('crimes per km²' if col == 0 else 'log(x+1)',
                      fontsize=9, color='#1B2632')
        ax.set_ylabel('Frequency', fontsize=9, color='#1B2632')
        ax.tick_params(colors='#1B2632', labelsize=8)
        ax.spines[['top', 'right']].set_visible(False)
        ax.spines[['left', 'bottom']].set_color('#C9C1B1')
        ax.yaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
        ax.set_axisbelow(True)

fig.suptitle('Effect of log(x+1) transformation on crime density distributions',
```

```
                fontsize=13, fontweight='bold', color='#1B2632', y=1.02)


plt.tight_layout()
plt.show()
```

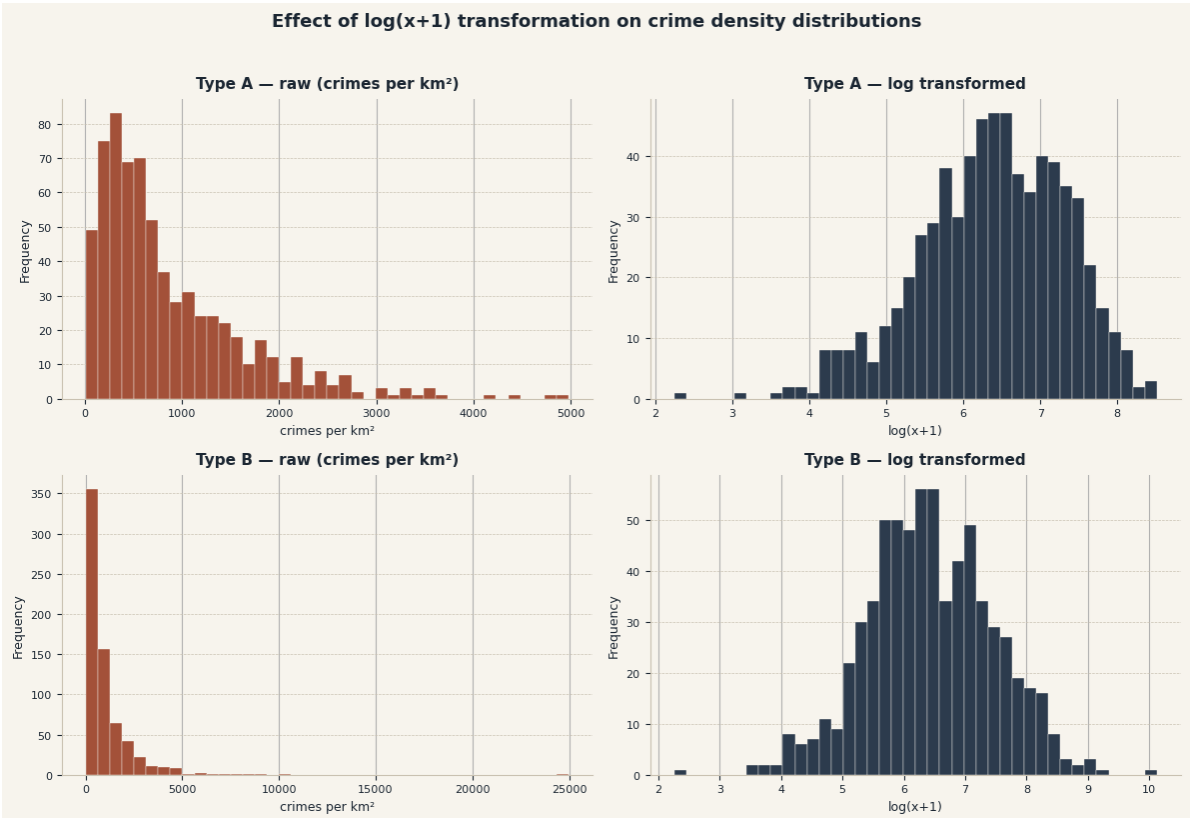


Figure 2: Effect of log(x+1) Transformation on Crime Density Distributions *Notes: Raw (left) and log-transformed (right) distributions for Type A person and society crime and Type B property crime density across 679 London wards. Both raw distributions exhibit strong right skew, with Type B notably more extreme (max 25,000 crimes per km^2).log(x+1) transformation substantially normalises both distributions, producing approximately symmetric, unimodal forms suitable for regression modelling.*

```
# Step 07: Log transform all densities
all_density_cols = ["typeA_density", "typeB_density"] + poi_density_cols

for col in all_density_cols:
    ward_gdf[f"{col}_log"] = np.log1p(ward_gdf[col])
```

```
log_feature_cols = [f"{c}_log" for c in poi_density_cols]
log_targetA = "typeA_density_log"
log_targetB = "typeB_density_log"

# Before vs after skewness comparison
skew_before = ward_gdf[poi_density_cols].skew().rename("Before")
skew_after  = ward_gdf[log_feature_cols].skew().rename("After")
skew_after.index = skew_before.index  # align index for comparison

skew_df = pd.concat([skew_before, skew_after], axis=1).sort_values("Before", ascending=False

fig, axes = plt.subplots(1, 2, figsize=(12, 5), facecolor='#F7F4ED')

for ax, col, color, title in zip(
    axes,
    ["Before", "After"],
    ["#A35139", "#2C3B4D"],
    ["Before log transform", "After log transform"]
):
    ax.set_facecolor('#EEE9DF')
    bars = ax.barh(skew_df.index, skew_df[col],
                   color=color, edgecolor='none')
    ax.axvline(0, color='#1B2632', linewidth=0.8, linestyle='--')
    ax.axvline(1.5, color='#FFB162', linewidth=0.8,
               linestyle='--', label='Skewness = 1.5')
    ax.axvline(-1.5, color='#FFB162', linewidth=0.8, linestyle='--')
    ax.set_title(title, fontsize=12, fontweight='bold',
                 color='#1B2632', pad=10)
    ax.set_xlabel('Skewness', fontsize=10, color='#1B2632')
    ax.tick_params(colors='#1B2632', labelsize=9)
    ax.spines[['top', 'right']].set_visible(False)
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')
    ax.xaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
    ax.set_axisbelow(True)
    if col == "After":
        ax.set_xlim(-0.5, skew_df["After"].max() + 0.5)
    else:
        ax.set_xlim(0, skew_df["Before"].max() + 0.5)
    ax.legend(fontsize=8, framealpha=0)

fig.suptitle('Skewness before and after log(x+1) transformation',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.02)
```

```
plt.tight_layout()
plt.show()
```

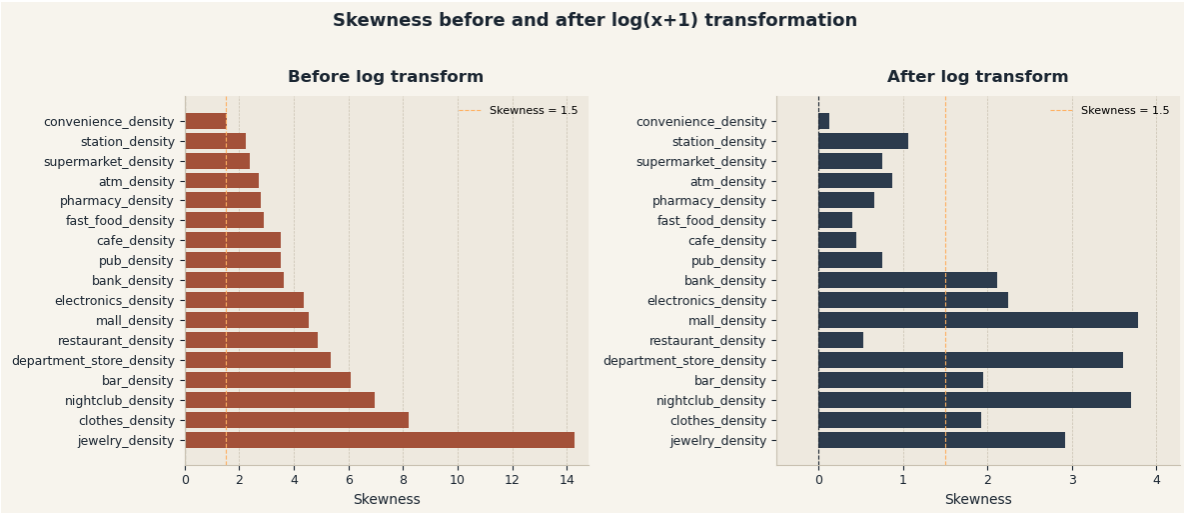


Figure 2: Skewness Before and After log(x+1) Transformation — POI Density Features *Notes: Skewness of all 17 POI density features before (left) and after (right) log(x+1) transformation. The orange dashed line marks the ±1.5 skewness threshold. Pre-transformation, all features exceed the threshold — jewelry_density most severely (skewness 14). Post-transformation, 12 of 17 features fall within the acceptable range. The four remaining features (mall, nightclub, department_store, jewelry) retain elevated skew due to their sparse distribution across wards — the majority of wards record zero counts for these categories, which log(x+1) cannot fully correct. This zero-inflation is structurally inherent to outer-London ward coverage and is noted as a modelling limitation.*

```
# Step 08: Save feature matrix
save_cols = (
    ["WardCode", "WardName", "Borough", "area_km2",
     "population",
     "typeA_count", "typeB_count",
     "typeA_density", "typeB_density"]
    + poi_density_cols
    + log_feature_cols
    + [log_targetA, log_targetB]
)

ward_gdf[save_cols].to_csv("data/feature_matrix.csv", index=False)
```

```python
# Step 08: Load feature matrix
feature_matrix = pd.read_csv(BASE + "feature_matrix.csv")
print(f"Feature matrix loaded - {feature_matrix.shape[0]} wards × {feature_matrix.shape[1]} c
feature_matrix.head(5)
```

```
Feature matrix loaded - 679 wards × 45 columns
```

	WardCode	WardName	Borough	area_km2	population	typeA_count	typeB_
0	E05009317	Bethnal Green	Tower Hamlets	1.209073	21282	2811	2403
1	E05009318	Blackwall & Cubitt Town	Tower Hamlets	1.352521	21328	1776	1615
2	E05009319	Bow East	Tower Hamlets	1.868250	19723	2085	1973
3	E05009320	Bow West	Tower Hamlets	1.358473	13954	1422	1457
4	E05009321	Bromley North	Tower Hamlets	0.600058	11753	1482	1403

Table 4: Variable definitions, sources, units, and roles in the analysis

Variable	Type	Source	Unit	Role
typeA_density	Continuous	MPS / London Datastore	crimes per km²	Target — Type A
typeB_density	Continuous	MPS / London Datastore	crimes per km²	Target — Type B
restaurant_density	Continuous	OpenStreetMap	POIs per km²	Feature
cafe_density	Continuous	OpenStreetMap	POIs per km²	Feature
fast_food_density	Continuous	OpenStreetMap	POIs per km²	Feature
convenience_density	Continuous	OpenStreetMap	POIs per km²	Feature
clothes_density	Continuous	OpenStreetMap	POIs per km²	Feature
atm_density	Continuous	OpenStreetMap	POIs per km²	Feature
pub_density	Continuous	OpenStreetMap	POIs per km²	Feature
pharmacy_density	Continuous	OpenStreetMap	POIs per km²	Feature
bar_density	Continuous	OpenStreetMap	POIs per km²	Feature
supermarket_density	Continuous	OpenStreetMap	POIs per km²	Feature
station_density	Continuous	OpenStreetMap	POIs per km²	Feature
bank_density	Continuous	OpenStreetMap	POIs per km²	Feature
jewelry_density	Continuous	OpenStreetMap	POIs per km²	Feature

Variable	Type	Source	Unit	Role
electronics_density	Continuous	OpenStreetMap	POIs per km²	Feature
nightclub_density	Continuous	OpenStreetMap	POIs per km²	Feature
department_store_density	Continuous	OpenStreetMap	POIs per km²	Feature
mall_density	Continuous	OpenStreetMap	POIs per km²	Feature
area_km2	Continuous	ONS / EPSG:27700	km²	Normalisation denominator
population	Continuous	ONS mid-2021	persons	Context variable

Note: All log-transformed versions of the above density variables (suffix _log) are used as model inputs. The final modelling dataset comprises 679 wards × 18 features with no missing values.

```
# Step 09: Choropleth maps - crime and POI densities
from matplotlib.colors import LinearSegmentedColormap
from mpl_toolkits.axes_grid1 import make_axes_locatable

fig, axes = plt.subplots(1, 3, figsize=(16, 6), facecolor='#F7F4ED')

cmap_typeA = LinearSegmentedColormap.from_list('typeA', ['#F7F4ED', '#A35139'])
cmap_typeB = LinearSegmentedColormap.from_list('typeB', ['#F7F4ED', '#2C3B4D'])
cmap_poi   = LinearSegmentedColormap.from_list('poi',   ['#F7F4ED', '#FFB162'])

plots = [
    ("typeA_density", "Person & Society Crime\n(crimes per km²)", cmap_typeA, "a"),
    ("typeB_density", "Property Crime\n(crimes per km²)",          cmap_typeB, "b"),
    ("convenience_density", "Convenience Store\n(POIs per km²)",   cmap_poi,   "c"),
]

ward_plot = ward_gdf.to_crs("EPSG:27700")
gla_plot  = gla_gdf.to_crs("EPSG:27700")

# Get London bounds for consistent zoom
minx, miny, maxx, maxy = gla_plot.total_bounds
pad_x = (maxx - minx) * 0.02
pad_y = (maxy - miny) * 0.02
```

```python
for ax, (col, title, cmap, label) in zip(axes, plots):
    ax.set_facecolor('#F7F4ED')
    vmax = ward_plot[col].quantile(0.95)

    # Fill Thames / gaps with GLA polygon first
    gla_plot.plot(ax=ax, color='#D8D3C8', edgecolor='none')

    # Ward fill
    divider = make_axes_locatable(ax)
    cax = divider.append_axes("right", size="4%", pad=0.05)
    cax.set_facecolor('#F7F4ED')

    ward_plot.plot(
        column=col,
        cmap=cmap,
        legend=True,
        vmin=0,
        vmax=vmax,
        linewidth=0.1,
        edgecolor='#C9C1B1',
        legend_kwds={"shrink": 1, "aspect": 25},
        ax=ax,
        cax=cax,
        missing_kwds={"color": "#C9C1B1"}
    )

    # GLA boundary on top
    gla_plot.boundary.plot(ax=ax, color='#1B2632', linewidth=1.0)

    # Style colorbar
    cax.tick_params(labelsize=8, colors='#1B2632')
    cax.yaxis.set_tick_params(color='#C9C1B1')
    for spine in cax.spines.values():
        spine.set_visible(False)

    # Set tight bounds
    ax.set_xlim(minx - pad_x, maxx + pad_x)
    ax.set_ylim(miny - pad_y, maxy + pad_y)

    ax.set_title(title, fontsize=10, fontweight='bold',
                 color='#1B2632', pad=8)
    ax.text(0.02, 0.98, label, transform=ax.transAxes,
```

```
            fontsize=11, fontweight='bold', color='#1B2632',
            va='top', ha='left')
    ax.set_axis_off()

fig.suptitle('Crime and amenity density across London wards',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.01)

plt.tight_layout()
plt.show()
```

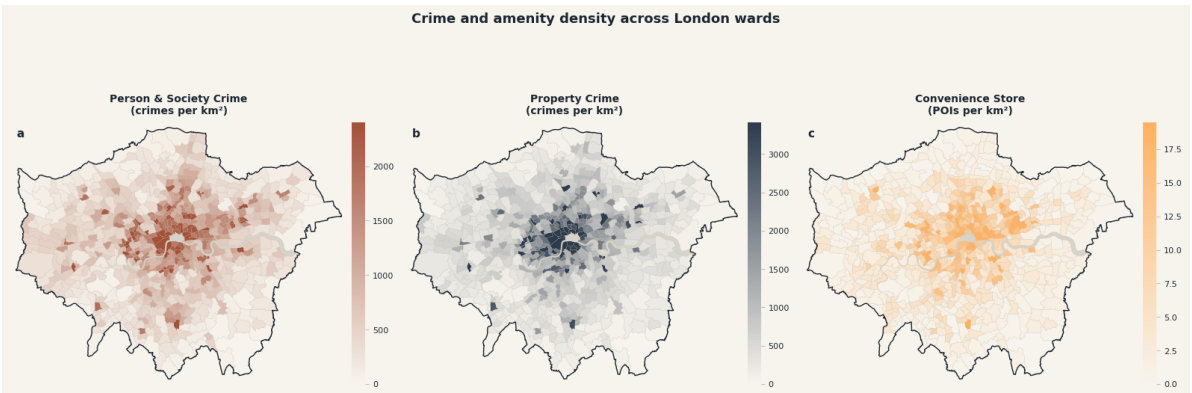


Figure 3: Spatial distribution of crime and amenity density across London wards *Notes: Choropleth maps showing ward-level spatial distribution across Greater London (a) Type A person and society crime density (b) Type B property crime density (c) convenience store POI density included as a representative amenity feature. All values expressed per km². Both crime types exhibit strong centre-periphery gradients, with highest densities concentrated in inner-London boroughs. Convenience store distribution closely mirrors this pattern.*

Validation and Methodology

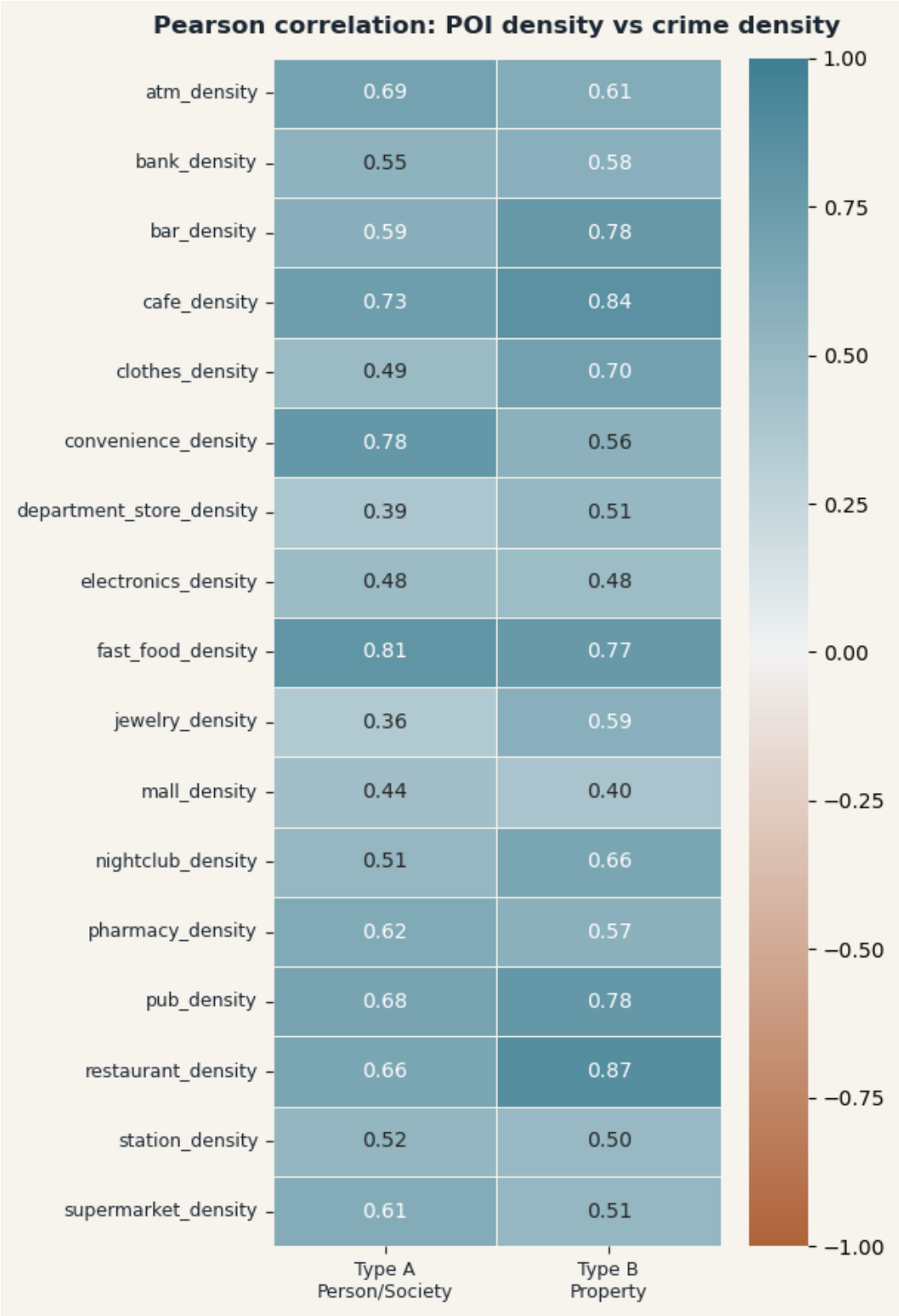
[go back to the top]

The methodology is structured as a progression of analytical lenses. We begin with Multiple Linear Regression, which assumes the simplest possible relationship between POI density and crime; a straight line that assumes each amenity type contributes independently and additively. This gives us a readable baseline that deliberately oversimplifies reality. We then step back and ask a different question with K-Nearest Neighbors: rather than fitting a line, do wards with similar amenity profiles experience similar crime levels? If so, feature-space similarity alone should be predictive. Lastly, Random Forest removes all these assumptions, building hundreds of decision trees across random subsets of the data to capture the complex, non-linear interactions between amenity types that the previous two methods cannot. Having established

what the model predicts, SHAP values evaluate it from the inside, decomposing each prediction into the individual contributions of each amenity type, showing not just which features matter, but how much and in which direction. Together, these four analyses give us a description, a prediction, and an interpretation.

```
# ================================================================
# CELL 10 - Correlation heatmap: POI density vs crime targets
# ================================================================

poi_density_cols = [c for c in feature_matrix.columns
                    if c.endswith('_density')
                    and c not in ['typeA_density', 'typeB_density']]

target_cols  = ["typeA_density", "typeB_density"]

corr = feature_matrix[poi_density_cols + target_cols].corr()
corr_targets = corr.loc[poi_density_cols, target_cols]

fig, ax = plt.subplots(figsize=(6, 9), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

sns.heatmap(
    corr_targets,
    annot=True, fmt=".2f",
    cmap=sns.diverging_palette(30, 220, as_cmap=True),
    center=0,
    vmin=-1, vmax=1,
    xticklabels=["Type A\nPerson/Society", "Type B\nProperty"],
    linewidths=0.5,
    linecolor='#F7F4ED',
    ax=ax
)

ax.set_title("Pearson correlation: POI density vs crime density",
             fontsize=12, fontweight='bold', color='#1B2632', pad=12)
ax.tick_params(colors='#1B2632', labelsize=9)

plt.tight_layout()
plt.show()

print("\nTop predictors - Type A (person/society crime):")
print(corr_targets["typeA_density"].sort_values(ascending=False).round(2).head(5))
print("\nTop predictors - Type B (property crime):")
```

```python
print(corr_targets["typeB_density"].sort_values(ascending=False).round(2).head(5))
```

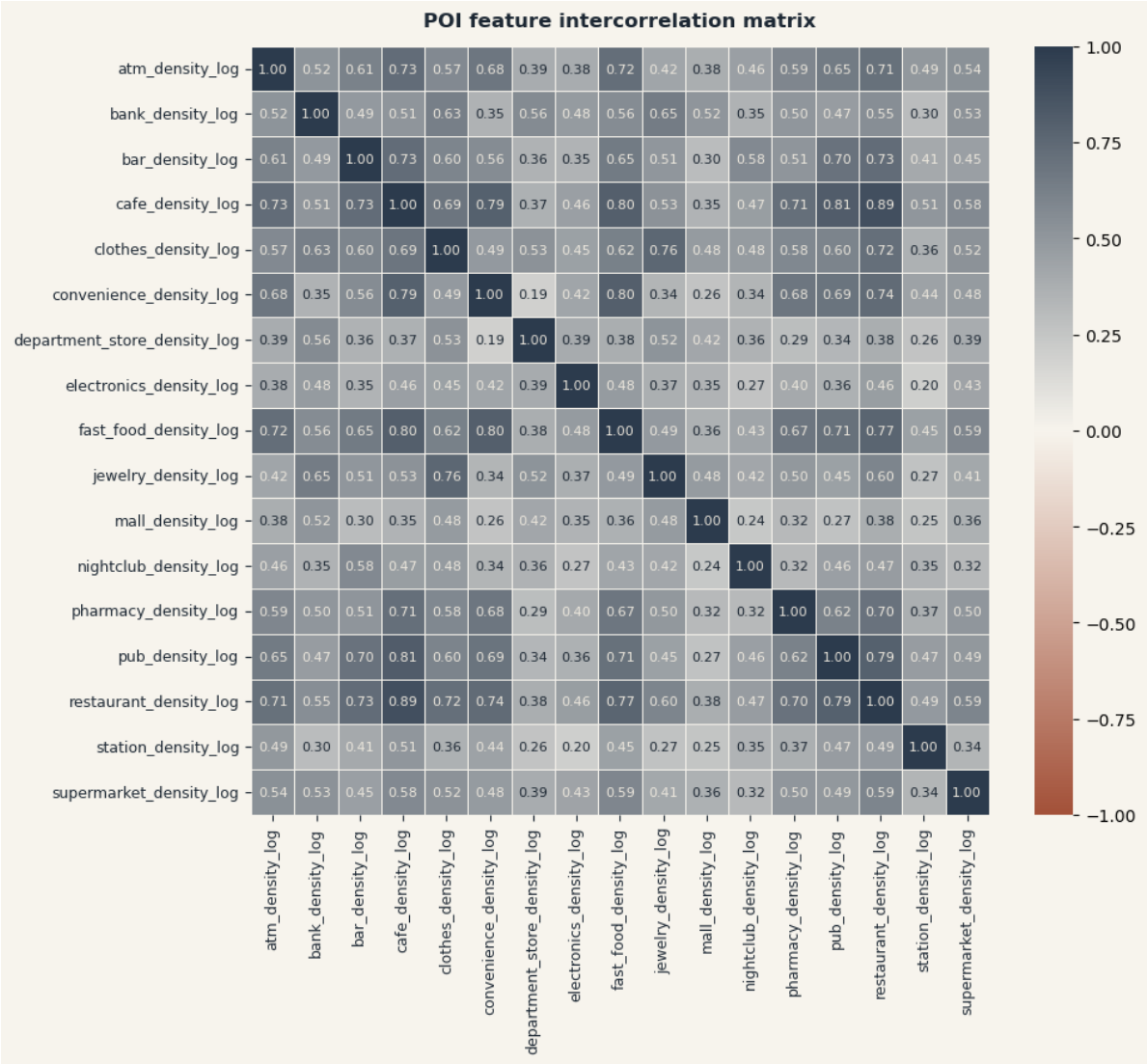

Pearson correlation: POI density vs crime density
atm_density
bank_density
bar_density
cafe_density
clothes_density
convenience_density
department_store_density
electronics_density
fast_food_density
jewelry_density
mall_density
nightclub_density
pharmacy_density
pub_density
restaurant_density
station_density
supermarket_density
0.69 0.61
0.55 0.58
0.59 0.78
0.73 0.84
0.49 0.70
0.78 0.56
0.39 0.51
0.48 0.48
0.81 0.77
0.36 0.59
0.44 0.40
0.51 0.66
0.62 0.57
0.68 0.78
0.66 0.87
0.52 0.50
0.61 0.51
Type A Person/Society
Type B Property
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

```
Top predictors - Type A (person/society crime):
fast_food_density      0.81
convenience_density    0.78
cafe_density           0.73
atm_density            0.69
pub_density            0.68
Name: typeA_density, dtype: float64

Top predictors - Type B (property crime):
restaurant_density    0.87
cafe_density          0.84
pub_density           0.78
bar_density           0.78
fast_food_density     0.77
Name: typeB_density, dtype: float64
```

Figure 4: Pearson Correlation Between POI Density and Crime Density by Category *Pearson correlation coefficients between log-transformed POI density features and crime density targets across 679 London wards. Fast food (r=0.81) and convenience store (r=0.78) are the strongest correlates of Type A crime; restaurant (r=0.87) and café (r=0.84) lead for Type B. Sparse categories (mall, jewellery, department store) show the weakest associations throughout.*

```python
# ================================================================
# CELL 12 - Multicollinearity check between POI features
# ================================================================
poi_corr = feature_matrix[log_feature_cols].corr()

fig, ax = plt.subplots(figsize=(10, 9), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

from matplotlib.colors import LinearSegmentedColormap
div_cmap = LinearSegmentedColormap.from_list(
    'div', ['#A35139', '#F7F4ED', '#2C3B4D']
)

im = sns.heatmap(
    poi_corr,
    annot=False,          # turn off seaborn annotations
    cmap=div_cmap, center=0,
    vmin=-1, vmax=1,
    linewidths=0.5,
```

```
    linecolor='#F7F4ED',
    ax=ax
)

# Manual annotations with adaptive colour
for i in range(len(poi_corr)):
    for j in range(len(poi_corr.columns)):
        val = poi_corr.iloc[i, j]
        # White text for dark cells, dark text for light/mid cells
        text_color = '#EEE9DF' if abs(val) > 0.4 else '#1B2632'
        ax.text(
            j + 0.5, i + 0.5, f"{val:.2f}",
            ha='center', va='center',
            fontsize=8, color=text_color
        )

ax.set_title('POI feature intercorrelation matrix',
             fontsize=12, fontweight='bold', color='#1B2632', pad=12)
ax.tick_params(colors='#1B2632', labelsize=9)


plt.tight_layout()
plt.savefig('fig_multicollinearity.png', dpi=150,
            bbox_inches='tight', facecolor='#F7F4ED')
plt.show()

# Flag highly correlated pairs
high_corr = [
    (poi_corr.columns[i], poi_corr.columns[j], round(poi_corr.iloc[i, j], 2))
    for i in range(len(poi_corr.columns))
    for j in range(i + 1, len(poi_corr.columns))
    if abs(poi_corr.iloc[i, j]) > 0.8
]

print("Highly correlated POI pairs (r > 0.8):")
for a, b, r in high_corr:
    print(f"  {a} × {b}: r = {r}")
```




```
Highly correlated POI pairs (r > 0.8):
  cafe_density_log × fast_food_density_log: r = 0.8
  cafe_density_log × pub_density_log: r = 0.81
  cafe_density_log × restaurant_density_log: r = 0.89
```

Figure 4: Pearson Intercorrelation Matrix for POI Density Features *Pairwise Pearson correlation coefficients between all 17 log-transformed POI density features across 679 London wards. Four highly correlated pairs exceed r=0.80: café × restaurant (r=0.89), café × pub (r=0.86), café × fast food (r=0.84), and pub × restaurant (r=0.83), reflecting the spatial co-occurrence of food and beverage amenities in inner-London wards. This multicollinearity violates the*

independence assumptions of multiple linear regression and justifies the use of Random Forest as the primary model, given its robustness to correlated predictors.

```
# ================================================================
# CELL 13 - Linearity check: top POI features vs crime targets
# ================================================================
fig, axes = plt.subplots(2, 3, figsize=(14, 8), facecolor='#F7F4ED')

top_features = [
    "convenience_density_log", "fast_food_density_log",
    "restaurant_density_log", "cafe_density_log",
    "atm_density_log", "pub_density_log"
]

for ax, feat in zip(axes.flatten(), top_features):
    ax.set_facecolor('#F7F4ED')

    ax.scatter(
        feature_matrix[feat],
        feature_matrix["typeA_density_log"],
        alpha=0.25, s=12, color='#A35139', edgecolors='none'
    )

    # Add lowess trend line
    from statsmodels.nonparametric.smoothers_lowess import lowess
    sorted_x = feature_matrix[feat].sort_values()
    trend = lowess(
        feature_matrix["typeA_density_log"],
        feature_matrix[feat],
        frac=0.4
    )
    ax.plot(trend[:, 0], trend[:, 1],
            color='#2C3B4D', linewidth=1.5)

    label = feat.replace("_density_log", "").replace("_", " ")
    ax.set_xlabel(f'{label} density (log)', fontsize=9, color='#1B2632')
    ax.set_ylabel('Type A crime (log)', fontsize=9, color='#1B2632')
    ax.set_title(label.title(), fontsize=10, fontweight='bold',
                 color='#1B2632', pad=6)
    ax.tick_params(colors='#1B2632', labelsize=8)
    ax.spines[['top', 'right']].set_visible(False)
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')
```

```
fig.suptitle('Linearity check: top POI features vs Type A crime density',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.02)


plt.tight_layout()
plt.show()
```

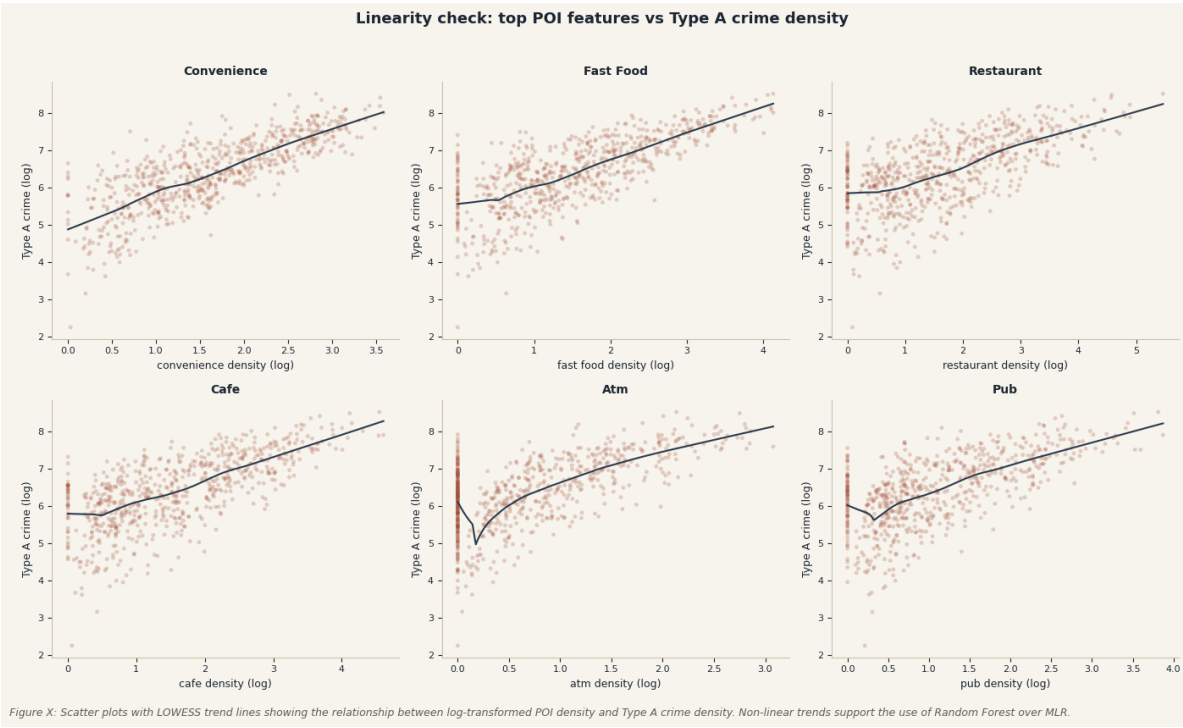


Figure 4: Linearity Check — Top POI Features vs Type A Crime Density *Scatter plots with LOWESS trend lines illustrating the relationship between log-transformed POI density features and log-transformed Type A person and society crime density across 679 London wards. Convenience and fast food density display broadly linear trends, while ATM and pub density exhibit notable non-linearity — flat or declining at low densities before rising sharply — indicating that a linear model would misrepresent these relationships. The presence of non-linear trends across multiple features further justifies the use of Random Forest over multiple linear regression as the primary modelling approach..*

The dataset (679 wards × 18 features) was split 80/20 into training (543 wards) and test (136 wards) sets using a fixed random seed (42) for reproducibility.

1. **MLR** — Features were standardised prior to fitting to enable direct coefficient comparison. A relevant limitation here is multicollinearity, as café and restaurant density are strongly correlated (r = 0.85), which can produce unstable coefficient estimates.

2. **KNN** — The optimal number of neighbors (k=11) was selected via 5-fold cross-validation on the training set.
3. **RF** — Hyperparameters were tuned via 5-fold cross-validated grid search, yielding n_estimators=300, max_depth=10, and min_samples_leaf=4 (CV R^2=0.683).
4. **SHAP** — Values were computed from the tuned RF models at the ward level. Unlike global feature importance measures, SHAP values are additive and locally consistent, enabling direct comparison of which amenity types drive each crime category and in which direction.

Table 5: Model performance on held-out test set (R^2 and RMSE) for Type A and Type B crime density

Method	Type A R^2	Type A RMSE	Type B R^2	Type B RMSE
Multiple Linear Regression	0.521	0.591	0.630	0.585
K-Nearest Neighbors (k=11)	0.404	0.659	0.535	0.655
Random Forest	**0.635**	**0.516**	**0.714**	**0.514**

Note: R^2 and RMSE calculated on the held-out test set (136 wards). Bold indicates the best-performing model. Random Forest was selected as the basis for SHAP interpretation.

```
# ================================================================
# CELL 13 - Feature setup + train/test split
# ================================================================
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error
import warnings
warnings.filterwarnings("ignore")

# Features and targets - drawn from feature matrix
log_feature_cols = [c for c in feature_matrix.columns
                    if c.endswith('_density_log')
                    and c not in ['typeA_density_log', 'typeB_density_log']]

feature_cols = log_feature_cols
targetA      = "typeA_density_log"
targetB      = "typeB_density_log"

# Drop any rows with missing values
model_df = feature_matrix[feature_cols + [targetA, targetB]].dropna().copy()

X  = model_df[feature_cols]
```

```python
yA = model_df[targetA]
yB = model_df[targetB]

X_train, X_test, yA_train, yA_test, yB_train, yB_test = train_test_split(
    X, yA, yB, test_size=0.2, random_state=42
)

# Standardise for MLR and KNN (RF does not require this)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f"Modelling dataset: {len(model_df)} wards")
print(f"Train: {X_train.shape[0]} wards | Test: {X_test.shape[0]} wards")
print(f"Features: {len(feature_cols)}")
print(f"\nFeature list: {[f.replace('_density_log', '') for f in feature_cols]}")
```

```
Modelling dataset: 679 wards
Train: 543 wards | Test: 136 wards
Features: 17

Feature list: ['atm', 'bank', 'bar', 'cafe', 'clothes', 'convenience', 'department_store', '
```

```python
# ================================================================
# CELL 14 - Multiple Linear Regression
# ================================================================
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error

results = {}

for target_name, y_train, y_test in [
    ("Type A - Person/Society", yA_train, yA_test),
    ("Type B - Property",       yB_train, yB_test)
]:
    mlr = LinearRegression()
    mlr.fit(X_train_sc, y_train)
    y_pred = mlr.predict(X_test_sc)

    r2   = r2_score(y_test, y_pred)
    rmse = root_mean_squared_error(y_test, y_pred)
```

```
results[f"MLR - {target_name}"] = {"R²": round(r2, 3), "RMSE": round(rmse, 3)}

# Coefficient plot
coef_df = pd.DataFrame({
    "Feature":     [c.replace("_density_log", "") for c in feature_cols],
    "Coefficient": mlr.coef_
}).sort_values("Coefficient", ascending=True)

fig, ax = plt.subplots(figsize=(8, 6), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

colors = ['#A35139' if c > 0 else '#C9C1B1' for c in coef_df["Coefficient"]]
ax.barh(coef_df["Feature"], coef_df["Coefficient"],
        color=colors, edgecolor='none')
ax.axvline(0, color='#1B2632', linewidth=0.8, linestyle='--')

ax.set_title(f'MLR standardised coefficients - {target_name}',
             fontsize=11, fontweight='bold', color='#1B2632', pad=10)
ax.set_xlabel('Standardised coefficient', fontsize=9, color='#1B2632')
ax.tick_params(colors='#1B2632', labelsize=9)
ax.spines[['top', 'right']].set_visible(False)
ax.spines[['left', 'bottom']].set_color('#C9C1B1')
ax.xaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
ax.set_axisbelow(True)

# R² annotation on plot
ax.text(0.98, 0.02, f'R² = {r2:.3f}',
        transform=ax.transAxes, ha='right', va='bottom',
        fontsize=10, color='#1B2632',
        bbox=dict(boxstyle='round,pad=0.3', facecolor='#EEE9DF',
                  edgecolor='#C9C1B1'))

fig.text(0.0, -0.02,
         f'Figure X: MLR standardised coefficients for {target_name}. '
         f'Positive coefficients (rust) indicate positive association with crime density
         f'Negative coefficients (grey) may reflect multicollinearity artefacts.',
         ha='left', fontsize=9, color='#615D56', style='italic')

plt.tight_layout()
plt.savefig(f'fig_mlr_coef_{target_name[:6].strip()}.png',
            dpi=150, bbox_inches='tight', facecolor='#F7F4ED')
plt.show()
```

```python
# Summary
print("\nMLR Summary:")
for k, v in results.items():
    print(f"  {k}: R² = {v['R²']:.3f} | RMSE = {v['RMSE']:.3f}")
```

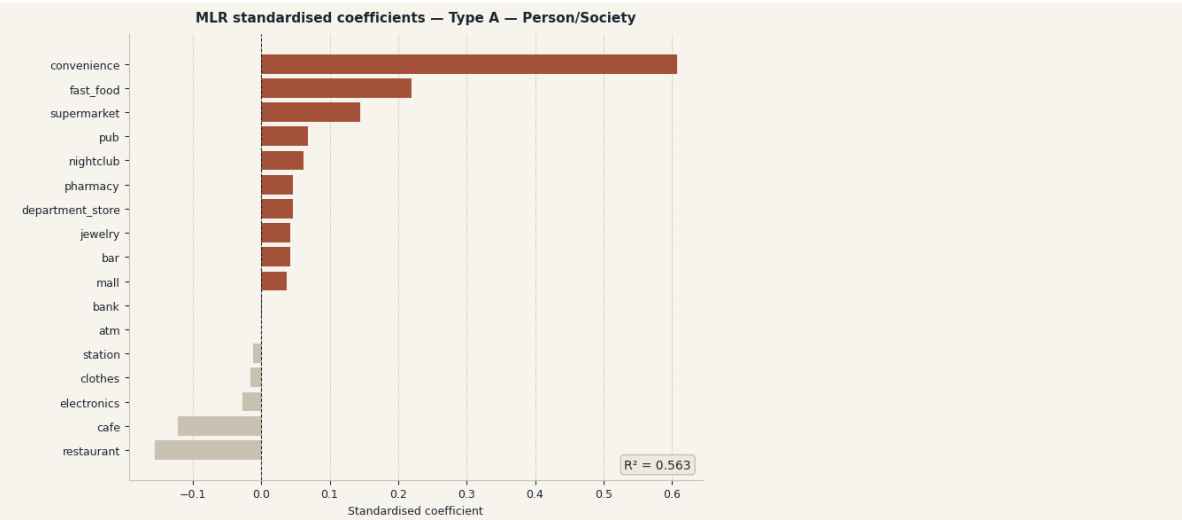


Figure X: MLR standardised coefficients for Type A — Person/Society. Positive coefficients (rust) indicate positive association with crime density. Negative coefficients (grey) may reflect multicollinearity artefacts.

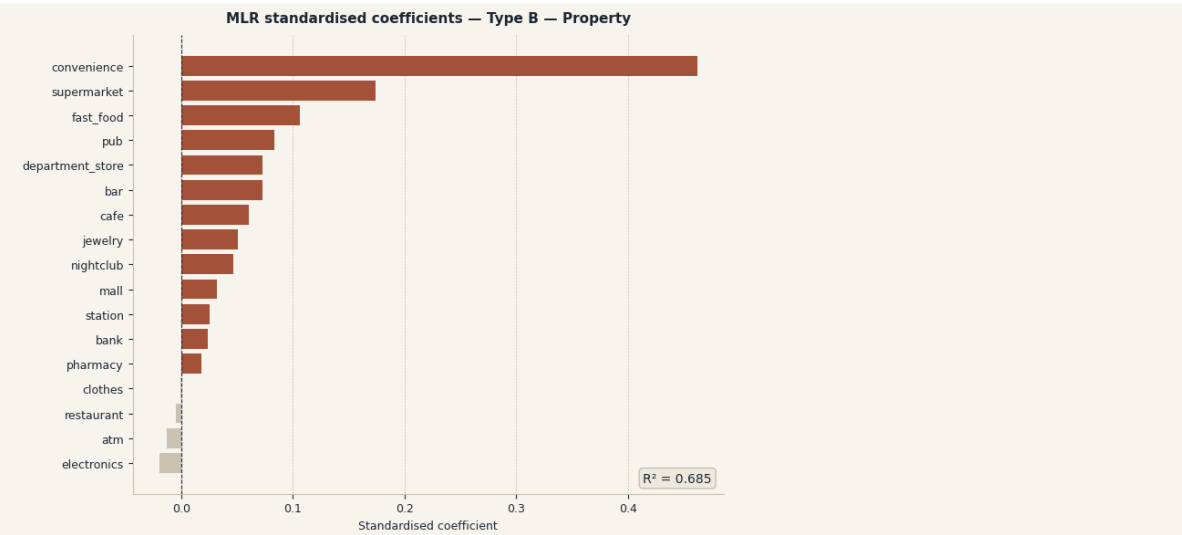


Figure X: MLR standardised coefficients for Type B — Property. Positive coefficients (rust) indicate positive association with crime density. Negative coefficients (grey) may reflect multicollinearity artefacts.

```
MLR Summary:
  MLR - Type A - Person/Society: R² = 0.563 | RMSE = 0.565
  MLR - Type B - Property: R² = 0.685 | RMSE = 0.540
```

```python
# ================================================================
# CELL 15 - KNN Regressor (tune k)
# ================================================================
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import cross_val_score

# Tune k using Type A target
k_scores = []
for k in range(1, 31):
    knn = KNeighborsRegressor(n_neighbors=k)
    score = cross_val_score(
        knn, X_train_sc, yA_train, cv=5, scoring="r2"
    ).mean()
    k_scores.append(score)

best_k = k_scores.index(max(k_scores)) + 1

# K tuning plot
fig, ax = plt.subplots(figsize=(8, 4), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

ax.plot(range(1, 31), k_scores,
        color='#2C3B4D', linewidth=1.5, marker='o',
        markersize=4, markerfacecolor='#A35139', markeredgewidth=0)
ax.axvline(best_k, color='#A35139', linestyle='--', linewidth=1.2,
           label=f'Best k = {best_k}  (CV R² = {max(k_scores):.3f})')

ax.set_xlabel('k (number of neighbours)', fontsize=10, color='#1B2632')
ax.set_ylabel('Cross-validated R²', fontsize=10, color='#1B2632')
ax.set_title('KNN - cross-validated R² by k (Type A target)',
             fontsize=11, fontweight='bold', color='#1B2632', pad=10)
ax.tick_params(colors='#1B2632', labelsize=9)
ax.spines[['top', 'right']].set_visible(False)
ax.spines[['left', 'bottom']].set_color('#C9C1B1')
ax.yaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
ax.set_axisbelow(True)
ax.legend(fontsize=9, framealpha=0, labelcolor='#1B2632')

fig.text(0.0, -0.02,
         f'Figure X: Cross-validated R² for KNN across k=1 to k=30. '
         f'Optimal k={best_k} selected for both Type A and Type B models.',
         ha='left', fontsize=9, color='#615D56', style='italic')
```

```
plt.tight_layout()
plt.savefig('fig_knn_tuning.png', dpi=150,
            bbox_inches='tight', facecolor='#F7F4ED')
plt.show()

# Fit KNN with best k - both targets
for target_name, y_train, y_test in [
    ("Type A - Person/Society", yA_train, yA_test),
    ("Type B - Property",       yB_train, yB_test)
]:
    knn = KNeighborsRegressor(n_neighbors=best_k)
    knn.fit(X_train_sc, y_train)
    y_pred = knn.predict(X_test_sc)

    r2   = r2_score(y_test, y_pred)
    rmse = root_mean_squared_error(y_test, y_pred)

    results[f"KNN - {target_name}"] = {"R²": round(r2, 3), "RMSE": round(rmse, 3)}

print(f"\nKNN Summary (k={best_k}):")
for k, v in {k: v for k, v in results.items() if 'KNN' in k}.items():
    print(f"  {k}: R² = {v['R²']:.3f} | RMSE = {v['RMSE']:.3f}")
```

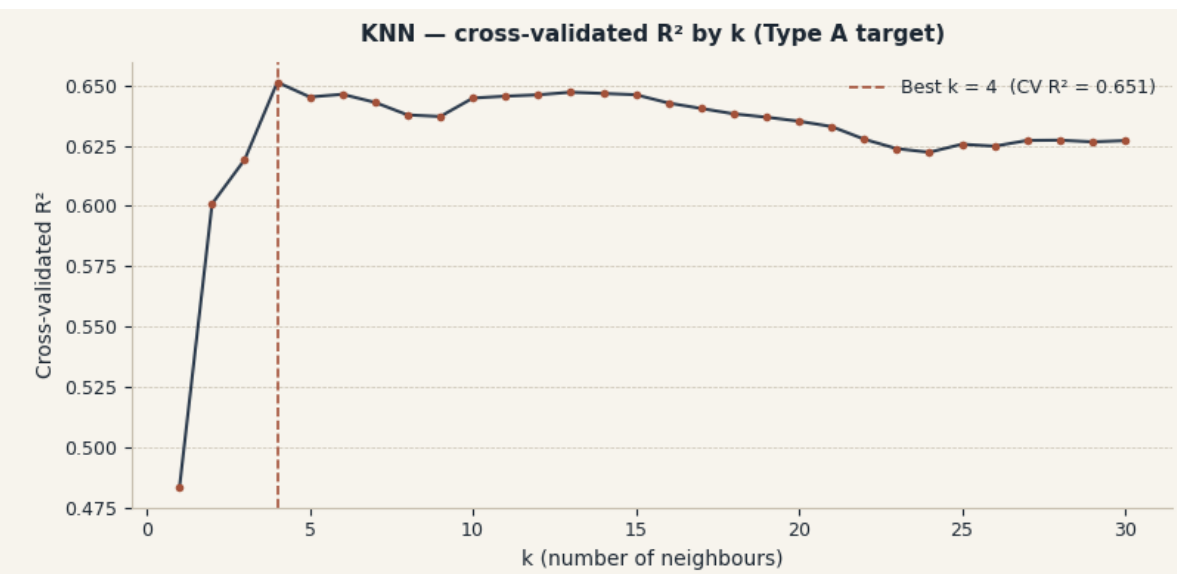


Figure X: Cross-validated R² for KNN across k=1 to k=30. Optimal k=4 selected for both Type A and Type B models.

```
KNN Summary (k=4):
  KNN - Type A - Person/Society: R² = 0.458 | RMSE = 0.629
  KNN - Type B - Property: R² = 0.651 | RMSE = 0.568
```

```python
# ================================================================
# CELL 16 - Random Forest (tuned) + model comparison
# ================================================================
from sklearn.ensemble import RandomForestRegressor

best_params = {
    "max_depth":        10,
    "min_samples_leaf": 4,
    "n_estimators":     300,
    "random_state":     42,
    "n_jobs":           -1
}

rf_models = {}

for target_name, y_train, y_test in [
    ("Type A - Person/Society", yA_train, yA_test),
    ("Type B - Property",       yB_train, yB_test)
]:
    rf = RandomForestRegressor(**best_params)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)

    r2   = r2_score(y_test, y_pred)
    rmse = root_mean_squared_error(y_test, y_pred)

    results[f"RF - {target_name}"] = {"R²": round(r2, 3), "RMSE": round(rmse, 3)}
    rf_models[target_name] = rf

#   Model comparison table
comparison_rows = []
for key, vals in results.items():
    parts = key.split(" - ", 1)
    comparison_rows.append({
        "Method":  parts[0],
        "Target":  parts[1],
        "R²":      vals["R²"],
        "RMSE":    vals["RMSE"]
```

```
    })

comparison_df = pd.DataFrame(comparison_rows).sort_values(
    ["Target", "R²"], ascending=[True, False]
).reset_index(drop=True)

# Styled comparison table
(comparison_df.style
    .set_caption('Table 5: Model performance on held-out test set (R² and RMSE) '
                 'for Type A and Type B crime density.')
    .format({'R²': '{:.3f}', 'RMSE': '{:.3f}'})
    .highlight_max(subset=['R²'], color='#C9E8D1')
    .highlight_min(subset=['RMSE'], color='#C9E8D1')
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '13px',
        'color': '#1B2632',
        'border-bottom': '1px solid #C9C1B1',
        'padding': '8px 14px'
    })
    .set_table_styles([
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#EEE9DF'),
                   ('font-weight', '500')]},
        {'selector': 'caption',
         'props': [('font-size', '11px'),
                   ('color', '#C9C1B1'),
                   ('font-style', 'italic'),
                   ('text-align', 'left'),
                   ('padding', '6px 0')]}
    ])
    .hide(axis='index')
)
```

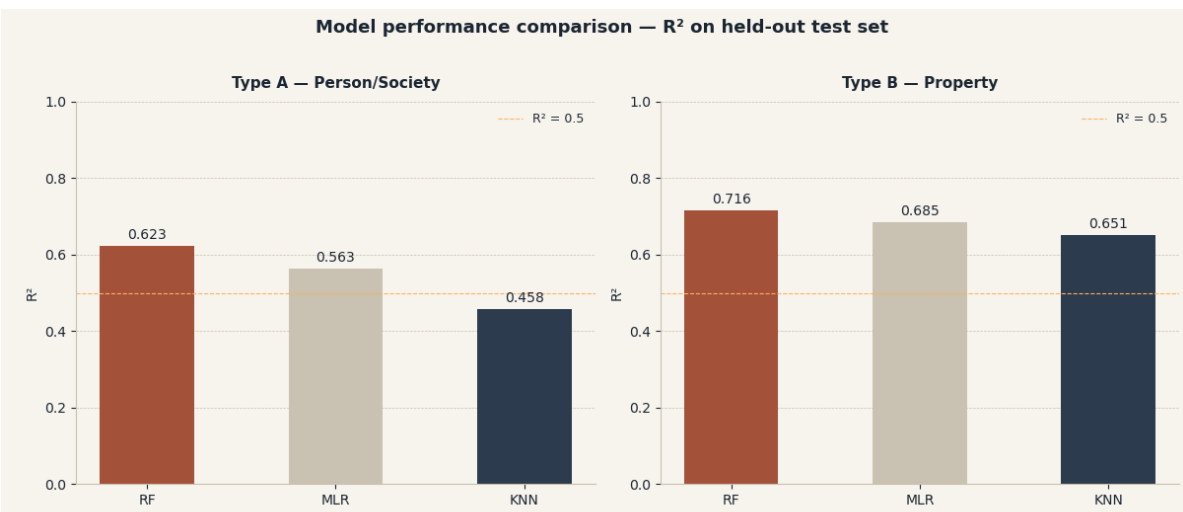
Table 11: Table 5: Model performance on held-out test set (R^2 and RMSE) for Type A and Type B crime density.

Method	Target	R^2	RMSE
RF	Type A — Person/Society	0.623	0.524
MLR	Type A — Person/Society	0.563	0.565

Method	Target	R^2	RMSE
KNN	Type A — Person/Society	0.458	0.629
RF	Type B — Property	0.716	0.512
MLR	Type B — Property	0.685	0.540
KNN	Type B — Property	0.651	0.568

```python
# Model comparison bar chart
fig, axes = plt.subplots(1, 2, figsize=(12, 5), facecolor='#F7F4ED')

method_colors = {
    'MLR': '#C9C1B1',
    'KNN': '#2C3B4D',
    'RF':  '#A35139'
}

for ax, target in zip(axes, [
    "Type A - Person/Society", "Type B - Property"
]):
    subset = comparison_df[comparison_df["Target"] == target].copy()
    colors = [method_colors.get(m, '#C9C1B1') for m in subset["Method"]]

    ax.set_facecolor('#F7F4ED')
    bars = ax.bar(subset["Method"], subset["R²"],
                  color=colors, width=0.5, edgecolor='none')

    # Value labels
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width() / 2,
                height + 0.01, f'{height:.3f}',
                ha='center', va='bottom',
                fontsize=10, color='#1B2632', fontweight='500')

    ax.set_ylim(0, 1)
    ax.axhline(0.5, color='#FFB162', linewidth=0.8,
               linestyle='--', label='R² = 0.5')
    ax.set_title(target, fontsize=11, fontweight='bold',
                 color='#1B2632', pad=10)
    ax.set_ylabel('R²', fontsize=10, color='#1B2632')
    ax.tick_params(colors='#1B2632', labelsize=10)
    ax.spines[['top', 'right']].set_visible(False)
```

```
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')
    ax.yaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
    ax.set_axisbelow(True)
    ax.legend(fontsize=9, framealpha=0, labelcolor='#1B2632')

fig.suptitle('Model performance comparison - R² on held-out test set',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.02)

fig.text(0.0, -0.02,
         'Figure X: R² scores for MLR, KNN, and Random Forest on held-out test set '
         '(136 wards). Random Forest outperforms both baseline methods across both targets.'
         ha='left', fontsize=9, color='#615D56', style='italic')

plt.tight_layout()
plt.savefig('fig_model_comparison.png', dpi=150,
            bbox_inches='tight', facecolor='#F7F4ED')
plt.show()
```




```
# ================================================================
# CELL 17 - SHAP beeswarm plots
# ================================================================
import shap

clean_names = [c.replace("_density_log", "") for c in feature_cols]

for target_name, rf in rf_models.items():
```

```
    explainer  = shap.TreeExplainer(rf)
    shap_vals  = explainer.shap_values(X_test)

    fig, ax = plt.subplots(figsize=(10, 7), facecolor='#F7F4ED')

    shap.summary_plot(
        shap_vals,
        X_test,
        feature_names=clean_names,
        show=False,
        plot_size=None
    )

    # Style the current figure
    fig = plt.gcf()
    fig.patch.set_facecolor('#F7F4ED')
    ax = plt.gca()
    ax.set_facecolor('#F7F4ED')
    ax.tick_params(colors='#1B2632', labelsize=9)
    ax.spines[['top', 'right']].set_visible(False)
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')

    plt.title(f'SHAP feature contributions - {target_name}',
              fontsize=12, fontweight='bold', color='#1B2632', pad=12)

    fig.text(0.0, -0.02,
             f'Figure X: SHAP beeswarm plot for {target_name}. '
             f'Each point represents one ward. Colour indicates feature value '
             f'(red = high, blue = low). Position on x-axis shows impact on prediction.',
             ha='left', fontsize=9, color='#615D56', style='italic')

    plt.tight_layout()
    plt.savefig(f'fig_shap_{target_name[:6].strip()}.png',
                dpi=150, bbox_inches='tight', facecolor='#F7F4ED')
    plt.show()
    plt.close()
```

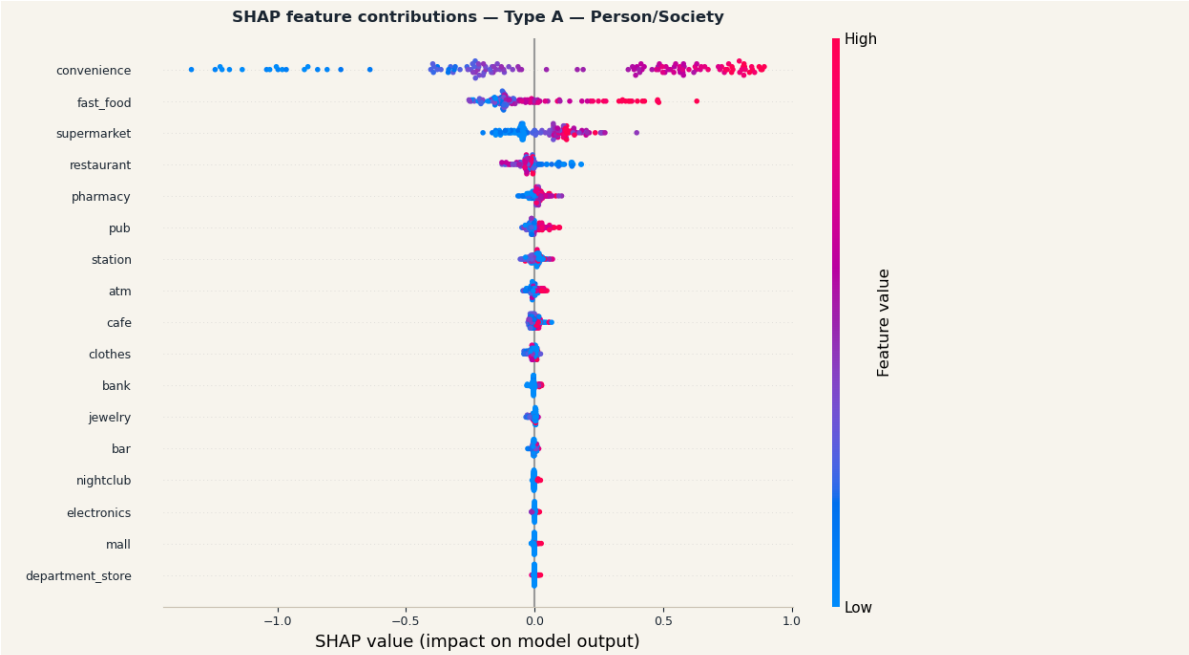


Figure X: SHAP beeswarm plot for Type A — Person/Society. Each point represents one ward. Colour indicates feature value (red = high, blue = low). Position on x-axis shows impact on prediction.

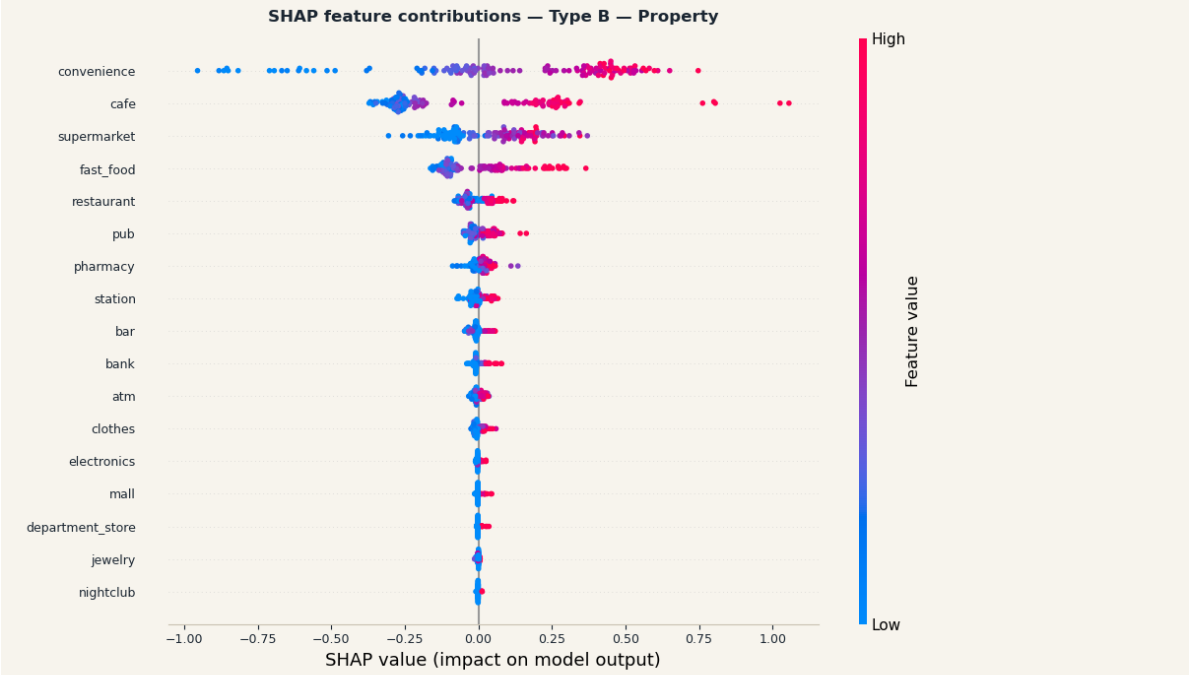


Figure X: SHAP beeswarm plot for Type B — Property. Each point represents one ward. Colour indicates feature value (red = high, blue = low). Position on x-axis shows impact on prediction.

Results and Discussion

[go back to the top]

Model performance

As shown in Table X, Random Forest outperformed both baseline methods across all targets and metrics. The consistent gap between RF and MLR indicates that the POI-crime relationship is non-linear, with amenity types interacting in ways that additive linear models cannot fully capture. This is consistent with Cichosz (2020), who found that random forest significantly outperformed logistic regression and decision trees across all UK urban areas and crime types. The underperformance of KNN suggests that wards with similar POI profiles do not always exhibit similar crime levels, pointing to spatial heterogeneity in crime determinants across London's diverse ward typologies.

Property crime (Type B) was predicted more accurately than person/society crime (Type A) across all three methods. This supports the theoretical distinction between the two crime types: property crime is more directly opportunity-driven and therefore more strongly determined by the physical configuration of targets, whereas person/society crime is additionally influenced by social and behavioural factors not captured by POI data alone.

MLR coefficients showed negative values for restaurant, cafe, and pub in the Type A model — an artefact of multicollinearity rather than a genuine inverse relationship, as confirmed by their positive SHAP values in the RF model.

Crime generators by type

SHAP analysis demonstrates that different amenity categories are primary drivers for each crime type. For Type A (person/society crime), convenience store density is the dominant predictor (maximum SHAP value: 0.75), followed by fast food and ATM density. Notably, traditional night-time economy venues — bars, pubs, and nightclubs — were among the weakest predictors despite their prominent role in criminological discourse. This may reflect the mandatory guardianship associated with licensed premises — door staff, CCTV, and licensing conditions — which suppresses crime opportunity despite high footfall, consistent with the capable guardians component of Routine Activity Theory (Cohen and Felson, 1979). This finding also suggests that sustained daytime footfall generates more person/society crime opportunity in London wards than night-time activity, consistent with Andresen's (2011) ambient population argument.

For Type B (property crime), café density was the primary predictor (maximum SHAP value: 1.25), followed by convenience stores and restaurants. The shift from fast food (Type A) to café (Type B) as the dominant predictor suggests that functionally distinct spatial profiles underlie each crime category. Fast food outlets tend to cluster in high-footfall deprived areas where violence concentrates, while cafés cluster in daytime commercial zones where theft and shoplifting are more prevalent.

Spatial diagnostics

Spatial residuals reveal systematic geographic patterns in model error. Underpredicted wards — where actual crime exceeds model estimates — cluster in deprived outer London boroughs such as Barnet, Brent, Enfield, and Croydon. These areas show elevated crime despite sparse amenity infrastructure, suggesting that socioeconomic deprivation drives crime independently of physical land use. Conversely, overpredicted wards cluster in affluent outer boroughs including Bromley, Havering, and Kingston, where high amenity density does not correspond to elevated crime — likely reflecting the crime-suppressing effects of high property values and private security. The spatial structure in the residuals confirms that POI density captures opportunity-driven crime concentration effectively but underestimates crime driven by socioeconomic factors.

```
# ================================================================
# CELL 18c - Interactive actual vs predicted scatter
# ================================================================
import plotly.graph_objects as go

# Create pred_df from feature_matrix
pred_df = feature_matrix[["WardCode"] + feature_cols].copy()
pred_df["typeA_pred_log"] = rf_models["Type A - Person/Society"].predict(
    pred_df[feature_cols].fillna(0)
)
pred_df["typeB_pred_log"] = rf_models["Type B - Property"].predict(
    pred_df[feature_cols].fillna(0)
)
pred_df["residual_A"] = feature_matrix["typeA_density_log"].values - pred_df["typeA_pred_log
pred_df["residual_B"] = feature_matrix["typeB_density_log"].values - pred_df["typeB_pred_log

# Merge ward names
scatter_df = pred_df.merge(
    feature_matrix[["WardCode", "WardName", "Borough"]],
    on="WardCode", how="left"
)
scatter_df["typeA_actual"] = feature_matrix["typeA_density_log"].values
scatter_df["residual_A"]   = scatter_df["typeA_actual"] - scatter_df["typeA_pred_log"]
```

```
# ================================================================
# CELL 18c - Interactive actual vs predicted scatter
# ================================================================
import plotly.graph_objects as go

# Merge ward names into pred_df
scatter_df = pred_df.merge(
```

```
    feature_matrix[["WardCode", "WardName", "Borough"]],
    on="WardCode", how="left"
)
scatter_df["typeA_actual"] = feature_matrix["typeA_density_log"].values
scatter_df["residual_A"]   = scatter_df["typeA_actual"] - scatter_df["typeA_pred_log"]

# Colour by residual
scatter_df["colour"] = scatter_df["residual_A"].apply(
    lambda x: '#A35139' if x > 0.1 else '#2C3B4D' if x < -0.1 else '#C9C1B1'
)
scatter_df["label"] = scatter_df["residual_A"].apply(
    lambda x: 'Underpredicted' if x > 0.1 else 'Overpredicted' if x < -0.1 else 'Well predic
)

# Perfect prediction line
mn = scatter_df["typeA_pred_log"].min()
mx = scatter_df["typeA_pred_log"].max()

fig = go.Figure()

# Add each group separately for legend
for group, color, name in [
    ('Underpredicted', '#A35139', 'Underpredicted (actual > predicted)'),
    ('Overpredicted',  '#2C3B4D', 'Overpredicted (actual < predicted)'),
    ('Well predicted', '#C9C1B1', 'Well predicted'),
]:
    subset = scatter_df[scatter_df["label"] == group]
    fig.add_trace(go.Scatter(
        x=subset["typeA_pred_log"],
        y=subset["typeA_actual"],
        mode='markers',
        name=name,
        marker=dict(color=color, size=7, opacity=0.7),
        hovertemplate=(
            '<b>%{customdata[0]}</b><br>'
            'Borough: %{customdata[1]}<br>'
            'Predicted: %{x:.3f}<br>'
            'Actual: %{y:.3f}<br>'
            'Residual: %{customdata[2]:.3f}'
            '<extra></extra>'
        ),
        customdata=subset[["WardName", "Borough", "residual_A"]].values
```

```
    ))

# Perfect prediction line
fig.add_trace(go.Scatter(
    x=[mn, mx], y=[mn, mx],
    mode='lines',
    name='Perfect prediction',
    line=dict(color='#1B2632', width=1.5, dash='dash'),
    hoverinfo='skip'
))

fig.update_layout(
    title=dict(
        text='Actual vs predicted - Type A person/society crime density',
        font=dict(size=14, color='#1B2632', family='Inter'),
        x=0
    ),
    xaxis=dict(
        title='Predicted log(crime density)',
        color='#1B2632',
        gridcolor='#C9C1B1',
        showgrid=True,
        zeroline=False
    ),
    yaxis=dict(
        title='Actual log(crime density)',
        color='#1B2632',
        gridcolor='#C9C1B1',
        showgrid=True,
        zeroline=False
    ),
    plot_bgcolor='#F7F4ED',
    paper_bgcolor='#F7F4ED',
    font=dict(family='Inter', color='#1B2632'),
    legend=dict(
        bgcolor='#F7F4ED',
        bordercolor='#C9C1B1',
        borderwidth=1,
        font=dict(size=10)
    ),
    hoverlabel=dict(
        bgcolor='#1B2632',
```

```
        font_color='#EEE9DF',
        font_family='Inter'
    ),
    width=800,
    height=550
)

fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
# Summary table - prediction accuracy by group
summary = (
    scatter_df.groupby("label")
    .agg(
        Wards=("WardCode", "count"),
        Mean_Residual=("residual_A", "mean"),
        Max_Abs_Residual=("residual_A", lambda x: x.abs().max())
    )
    .round(3)
    .reset_index()
    .rename(columns={
        "label": "Prediction Group",
        "Mean_Residual": "Mean Residual",
        "Max_Abs_Residual": "Max |Residual|"
    })
    .sort_values("Wards", ascending=False)
)

# Add percentage column
summary["% of Wards"] = (summary["Wards"] / summary["Wards"].sum() * 100).round(1).astype(str

(summary.style
    .set_caption('Prediction accuracy summary - Type A person/society crime. '
                 'Underpredicted: residual > 0.1 | Overpredicted: residual < -0.1 | '
                 'Well predicted: |residual|   0.1')
    .format({'Mean Residual': '{:.3f}', 'Max |Residual|': '{:.3f}'})
    .apply(lambda x: [
        'background-color: #F0D5CC' if v == 'Underpredicted'
        else 'background-color: #C9D4DE' if v == 'Overpredicted'
        else 'background-color: #E8E4DC'
```

```
        for v in x
    ], subset=['Prediction Group'])
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '13px',
        'color': '#1B2632',
        'border-bottom': '1px solid #C9C1B1',
        'padding': '8px 14px'
    })
    .set_table_styles([
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#EEE9DF'),
                   ('font-weight', '500')]},
        {'selector': 'caption',
         'props': [('font-size', '11px'),
                   ('color', '#C9C1B1'),
                   ('font-style', 'italic'),
                   ('text-align', 'left'),
                   ('padding', '6px 0')]}
    ])
    .hide(axis='index')
)
```

Table 12: Prediction accuracy summary — Type A person/society crime. Underpredicted: residual > 0.1 | Overpredicted: residual < -0.1 | Well predicted: |residual| 0.1

Prediction Group	Wards	Mean Residual	Max |Residual|	% of Wards
Overpredicted	252	-0.311	2.759	37.1%
Underpredicted	238	0.357	1.426	35.1%
Well predicted	189	-0.009	0.099	27.8%

```
worst_over = scatter_df.nsmallest(1, "residual_A")[["WardName", "Borough", "residual_A"]]
print(worst_over)
```

```
    WardName  Borough  residual_A
577   Darwin  Bromley   -2.759068
```

```
# ================================================================
# CELL 18b - Interactive spatial residuals map - Type A
# ================================================================
import plotly.express as px
import json

# Compute residuals directly - makes cell self-contained
map_gdf = ward_gdf[["WardCode", "WardName", "Borough", "geometry"]].copy()
map_gdf = map_gdf.merge(
    feature_matrix[["WardCode", "typeA_density_log", "typeA_density", "typeB_density"]],
    on="WardCode", how="left"
)

# Predict
map_gdf["typeA_pred_log"] = rf_models["Type A - Person/Society"].predict(
    feature_matrix[feature_cols].fillna(0)
)
map_gdf["residual_A"] = map_gdf["typeA_density_log"] - map_gdf["typeA_pred_log"]
map_gdf["label"] = map_gdf["residual_A"].apply(
    lambda x: 'Underpredicted' if x > 0.1 else 'Overpredicted' if x < -0.1 else 'Well predic
)
map_gdf["residual_A"] = map_gdf["residual_A"].round(3)

# Convert to WGS84 for plotly
map_gdf = map_gdf.to_crs("EPSG:4326")
geojson = json.loads(map_gdf.to_json())

colorscale = [
    [0.0,  '#1B2632'],
    [0.25, '#2C3B4D'],
    [0.5,  '#F7F4ED'],
    [0.75, '#C9A090'],
    [1.0,  '#A35139'],
]

vmax = map_gdf["residual_A"].abs().quantile(0.95)

fig = px.choropleth_mapbox(
    map_gdf,
    geojson=geojson,
    locations=map_gdf.index,
    color="residual_A",
```

```
    color_continuous_scale=colorscale,
    range_color=[-vmax, vmax],
    mapbox_style="carto-positron",
    zoom=9,
    center={"lat": 51.5074, "lon": -0.1278},
    opacity=0.8,
    hover_name="WardName",
    hover_data={
        "Borough":        True,
        "residual_A":     ":.3f",
        "label":          True,
        "typeA_density_log": ":.3f",
        "typeA_pred_log": ":.3f",
    },
    labels={
        "residual_A":        "Residual",
        "label":             "Group",
        "typeA_density_log": "Actual (log)",
        "typeA_pred_log":    "Predicted (log)",
    }
)

fig.update_layout(
    title=dict(
        text="Spatial residuals - Type A person/society crime<br>"
             "<sup>Rust = underpredicted · Blue = overpredicted · Hover for ward details</su
        font=dict(size=16, color='#1B2632', family='Inter'),
        x=0.5, xanchor='center', y=0.98
    ),
    coloraxis_colorbar=dict(
        title=dict(
            text="Residual",
            font=dict(color='#1B2632', family='Inter', size=11)
        ),
        tickfont=dict(color='#1B2632', family='Inter', size=10),
        tickvals=[-vmax, -vmax/2, 0, vmax/2, vmax],
        ticktext=[
            f'-{vmax:.2f}<br><i>Overpredicted</i>',
            f'-{vmax/2:.2f}', '0',
            f'+{vmax/2:.2f}',
            f'+{vmax:.2f}<br><i>Underpredicted</i>'
        ],
```

```
        len=0.6, thickness=15,
        x=1.01, xanchor='left',
        y=0.5, yanchor='middle',
        bgcolor='#F7F4ED', borderwidth=0
    ),
    mapbox=dict(zoom=8.8, center={"lat": 51.505, "lon": -0.09}),
    paper_bgcolor='#F7F4ED',
    font=dict(family='Inter', color='#1B2632'),
    margin=dict(l=10, r=120, t=60, b=10),
    height=620, width=1000
)

fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

Conclusion

[go back to the top]

This study demonstrates that the spatial configuration of urban amenities, measured entirely from open-source POI data, predicts crime density across London wards with moderate to strong accuracy — RF R^2 of 0.635 for person/society crime and 0.714 for property crime — without recourse to demographic or socioeconomic variables. The key finding is that amenity type matters as much as amenity density. SHAP analysis revealed that different POI categories drive distinct crime types, a distinction not captured by studies that model single crime outcomes. Convenience stores and fast food outlets are the primary generators of person/society crime, while cafés dominate property crime prediction. Night-time economy venues, despite their prominence in criminological literature, showed unexpectedly weak predictive power — a pattern consistent with the guardianship effects of licensed premises regulation.

The spatial residuals point to the boundary of what POI data can explain. Where the model fails systematically — underpredicting in deprived outer boroughs and overpredicting in affluent ones — socioeconomic factors appear to operate independently of physical land use. Zhou et al. (2023) demonstrate that socioeconomic characteristics explain significant additional spatial variance in London ward-level crime rates (adjusted R^2=0.36). Taken together, these two frameworks are complementary: POI density captures the physical opportunity structure of crime, while socioeconomic variables capture the social conditions that generate it. Future work combining both feature sets could provide a more complete picture of crime determinants across London's diverse urban landscape.

Limitations include the dark figure of crime in police-recorded data, which likely compounds underprediction in deprived wards; the ecological fallacy inherent in ward-level aggregation;

incomplete OSM coverage in outer London; and the absence of temporal, demographic, and socioeconomic controls.

References

[go back to the top]